

Understanding Code Understandability Improvements in Code Reviews

Delano Oliveira, Reydney Santos, Benedito de Oliveira, Martin Monperrus, Fernando Castor, and Fernanda Madeiral

Abstract—*Context:* Code understandability plays a crucial role in software development, as developers spend between 58% and 70% of their time reading source code. Improving code understandability can lead to enhanced productivity and save maintenance costs. *Problem:* Experimental studies aim to establish what makes code more or less understandable in a controlled setting, but ignore that what makes code easier to understand in the real world also depends on extraneous elements such as developers' background and project culture and guidelines. Not accounting for the influence of these factors may lead to results that are sound but have little external validity. *Goal:* We aim to investigate how developers improve code understandability during software development through code review comments. Our assumption is that code reviewers are specialists in code quality within a project. *Method and Results:* We manually analyzed 2,401 code review comments from Java open-source projects on GitHub and found that over 42% of all comments focus on improving code understandability, demonstrating the significance of this quality attribute in code reviews. We further explored a subset of 385 comments related to code understandability and identified eight categories of code understandability concerns, such as incomplete or inadequate code documentation, bad identifier, and unnecessary code. Among the suggestions to improve code understandability, 83.9% were accepted and integrated into the codebase. Among these, only two (less than 1%) ended up being reverted. We also identified types of patches that improve code understandability, ranging from simple changes (e.g., removing unused code) to more context-dependent improvements (e.g., replacing method calling chains by existing API). Finally, we investigated the potential coverage of four well-known linters to flag the identified code understandability issues. These linters cover less than 30% of these issues, although some of them could be easily added as new rules. *Implications:* Our findings motivate and provide practical insight for the construction of tools to make code more understandable, e.g., understandability improvements are rarely reverted and thus can be used as reliable training data for specialized ML-based tools. This is also supported by our dataset, which can be used to train such models. Finally, our findings can also serve as a basis to develop evidence-based code style guides. *Data Availability:* Our data is publicly available at <https://codeupcrc.github.io>.

Index Terms—Code understandability, code understandability smells, code review

1 INTRODUCTION

READING code is the main activity of software developers [1], with studies [2], [3] estimating that developers spend between 58% and 70% of their time understanding source code. That means that easy-to-read code can save time and money in software development. Code understandability is the extent to which a developer can comprehend and make sense of source code. It covers a number of aspects, such as naming, commenting, and structuring [4]. Empirical studies, e.g., [5], [6], [7], [8], [9], have compared several different ways of writing code to find which factors impact code understandability, e.g., abbreviated vs. complete word identifier names [10].

Laboratory studies on code understandability have as their main advantage the possibility to account for and control extraneous factors and objectively establish relationships between independent and response variables. However, they are not able to account for elements such as project culture and guidelines, and developers' background, which directly impact how developers understand code. Not accounting for the influence of these factors may lead to results that are sound but have little external validity. In this paper, our motivation is to unveil what developers care about regarding code understandability and how they improve it, in a real-world setting. Our key insight is that this knowledge is available through publicly available code reviews in open-source software, where code reviewers act as project specialists in code quality.

Code review is a process in software development where developers read and comment on code written by fellow developers in the same project [11], [12], [13]. In a typical code review scenario, code changes are proposed through pull requests (e.g., in projects hosted on GitHub) and then reviewed by developers who suggest improvements. Once the concerns are addressed and the improvements are performed, the pull request is approved for integration into the codebase. These suggestions might concern several code quality aspects, including understandability. In this paper, we examine code understandability improvements

Delano Oliveira is with the Federal University of Pernambuco, 50732-970 Recife, Brazil, and the Federal Institute of Pernambuco, 55540-000 Palmares, Brazil (e-mail: delanohelio@gmail.com).

Reydney Santos and Benedito de Oliveira are with the Federal University of Pernambuco, 50732-970 Recife, Brazil (e-mail: rbs8@cin.ufpe.br; bfao@cin.ufpe.br).

Martin Monperrus is with the KTH Royal Institute of Technology, 114 28 Stockholm, Sweden (e-mail: monperrus@kth.se).

Fernando Castor is with the University of Twente, 7522 NB Enschede, The Netherlands, and the Federal University of Pernambuco, 50732-970 Recife, Brazil (e-mail: f.castor@utwente.nl).

Fernanda Madeiral is with the Vrije Universiteit Amsterdam, 1081 HV Amsterdam, The Netherlands (e-mail: fer.madeiral@gmail.com).

Manuscript received 28 August 2023; revised 8 August 2024; accepted 16 August 2024. Recommended for acceptance by K. Blincoe

(Corresponding author: Delano Oliveira.)

Digital Object Identifier no. 10.1109/TSE.2024.3453783

grounded on real comments about code quality made during code review. More specifically, we investigate (i) what code understandability issues are reported by developers during code review, and (ii) what changes developers apply to address those concerns.

Our methodology is founded on systematic manual analysis of comments written by developers during code review in OSS repositories. We selected 363 Java open-source projects from GitHub that have an active code review process. In the context of pull requests on GitHub, developers can write general or inline comments. The latter are associated with specific lines of the source code under review. Because we want to identify where the reviewers' concerns reside and what source code changes were applied to address these concerns, we exclusively focused on *inline code review comments* in this study¹. We selected a representative sample of 2,401 code review comments for our study. We first manually analyzed all those code review comments and identified the ones concerning code understandability. From this process, we identified 1,012 code review comments related to code understandability improvements. Then, we analyzed these comments and the corresponding remediation changes that developers made to improve code understandability.

We found that more than 42% of all analyzed code review comments suggest code understandability improvements in scopes such as executable source code, code comments, and text literals. This finding highlights the significance of improving code understandability during code review, as suggested by Bacchelli and Bird [11] and Sadowski et al. [12]. We also analyzed in more depth a random sample of 385 code review comments suggesting code understandability improvements. We identified and labeled the concerns related to code understandability, hereafter referred to as "understandability smells", that were mentioned by reviewers in our sample of 385 code review comments. We discover eight categories of code understandability smells (e.g., INCOMPLETE OR INADEQUATE CODE DOCUMENTATION, BAD IDENTIFIER, UNNECESSARY CODE) in 16 places in the source code (e.g., Method, Call, Literal, Class). Out of these code review comments, 3.9% (323) of the suggestions made by the reviewers were accepted, indicating that developers generally agree with suggestions for improving code understandability. In addition, we performed a fine-grained analysis of actual improvements (the 323 accepted code review comments) and identified different types of patches that improve code understandability (e.g., modify an identifier to express the meaning or type of an element, remove non-referred constants and other code elements, add or remove horizontal spacing). While some of these patches can be automatically detected and fixed by linters (e.g., unused variables), others are more challenging to identify as they relate to natural language components (e.g., incomplete code documentation) or are dependent on the context of the project and code style choices (e.g., using boolean expressions instead of equivalent functional methods). Furthermore, 97.2% of the applied patches for code review comments were integrated into the codebase of the project and they are very rarely reverted (less than

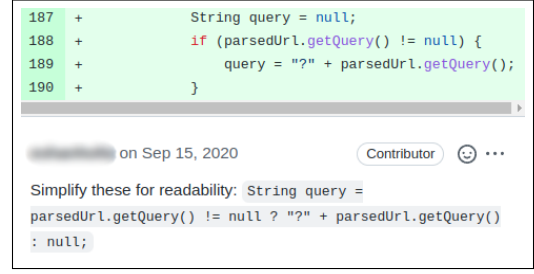


Fig. 1: Inline code review suggestion to replace an `if` statement by a ternary expression [14].

1%). The remaining 2.8% of patches had their code removed or replaced with a different solution. Finally, an examination of four popular linters shows that they can detect less than 30% of the analyzed code understandability issues. Some additional cases could be supported by the definition of new rules, whereas others would require them to also interact with natural language text (e.g., to detect typos). Our study provides practical guidance for the construction of tools to make code more understandable, e.g., some improvements that could be automated are currently not available in any of four popular linters, and understandability improvements are rarely reverted and thus can be used as reliable training data for specialized ML-based tools. This is also supported by our dataset, which can be used to train such models. Finally, our findings can serve as a basis to develop evidence-based code style guides. More broadly, this is a step in helping developers focus on more critical tasks during code review, such as verifying the correctness of code or identifying security vulnerabilities.

Data Availability. The resulting dataset of inline code review comments, which contains a classification of developers' intention with a focus on code understandability improvements and the resulting patches created to address them, is publicly available at <https://codeupcrc.github.io>.

2 BACKGROUND

This section provides a brief overview of the two concepts that underpin our investigation: code review and code understandability.

2.1 Code Review

Code review is an assessment of source code to identify a variety of quality issues, such as defects, security vulnerabilities, poor understandability, and bad design. In collaborative development platforms, code review takes place when source code changes are proposed to a project by a developer, the *submitter*. Then, the proposed changes are reviewed by other developers, the *reviewers*, who provide comments with, for instance, suggestions or requests for improvements that the submitter should apply to the code. Submitters and reviewers interact until the reviewers consider that the proposed code changes are good enough to be integrated into the codebase (else the proposed changes are rejected).

GitHub, the most popular collaborative software development platform², allows code review in the context of

1. Hereafter, we use the term code review comment for simplification.

2. <https://github.blog/2023-01-25-100-million-developers-and-counting/>

pull requests. Pull request reviewers can provide general code review comments in the body of the pull request timeline or *inline code review comments* that are specific to source code lines of the changed files. Inline comments are about fine-grained, localized reviewers' concerns, because they explicitly point to specific source code lines where the concerns lie. Figure 1 shows an example of an inline code review comment. The source code lines highlighted in green represent a change proposed to a GitHub project through a pull request. A reviewer wrote an inline comment associated with line 190, suggesting the replacement of the `if` block by a ternary expression. In such a case, the reviewer's concern is explicitly about readability.

In this paper, we leverage code review comments to investigate the reviewers' concerns related to code understandability and how the submitters address these concerns. Because we want to examine the solutions developers adopt at a fine level of granularity, e.g., the replacement of an `if` statement by a ternary expression (Figure 1), we focus on inline code review comments (we use the term code review comment for simplification). Since code reviewers are quality gatekeepers in software development, we hypothesize that, if code understandability is an important concern in practice, there are code review comments about it.

2.2 Code Understandability

Legibility and readability are both important factors for making code easy to read, but understandability is the ultimate goal. Legibility refers to how easily individual code elements can be distinguished from each other, while readability refers to how easily developers can comprehend the code as a whole [4]. Ultimately, understandability encompasses both legibility and readability and refers to how easily developers can extract relevant information from the source code, including non-functional parts, such as documentation, to perform software development or maintenance tasks [15]. In this work, we are interested in understandability.

In the literature, many studies have evaluated how to improve code legibility and readability [7], [8], [16], [17], [18], [19]. All these studies are based on an informal notion of *understandability improvement*. A code understandability improvement is a refactoring that, when applied to a program, makes its source code better at communicating to the developer what it does, according to one of more quality criteria [4].

A basic example of this type of change is renaming. Renaming a program element, in general, is not a functional correction and does not change the functionality of the program. However, a change to the name of a program element may make it clearer for other humans, by better communicating purpose or rationale. The main motivation for renaming is to improve code understandability, although there are some exceptions where renaming is not motivated by understandability, such as when it matches a reflection pattern. Moreover, certain aspects of source code are pertinent to understandability by nature, specifically documentation and formatting. Although disregarded by compilers, these elements are deliberately integrated into the source code to assist developers in comprehending the code.

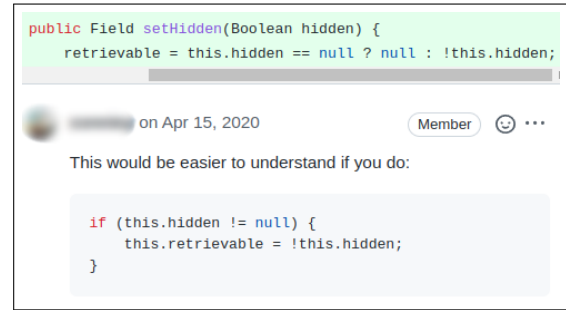


Fig. 2: Inline code review suggestion to replace a ternary expression by an `if` statement [22].

Many factors in the source code can affect code understandability, such as the number of identifiers, size of a line of code, and nesting [20], [21]. However, the definition of whether a change is an understandability improvement depends on the developer's background (e.g., skill and experience) [15]. Also, we could consider this can be affected by the developer's goals (e.g., implementing a new feature, writing tests, and fixing a bug). In the literature, for example, Gopstein et al. [18] found that subjects misunderstand the output of a code snippet when it contains ternary/conditional expression (e.g., `a?b:c`) instead of an `if` statement. In a similar study, Langhout and Aniche [9] did not find that the ternary operator causes misunderstanding. In a study by Medeiros et al. [7], the practitioners answered that ternary/conditional expression does not influence code understandability positively or negatively.

The perception of what understandability improvement is depends on the context in the real world. During code review, reviewers suggest source code changes to improve code understandability, but the development team can influence what is considered an improvement. For instance, in Figure 1, a reviewer suggested replacing an `if` statement by a ternary expression to improve code understandability. In Figure 2, which refers to another open-source project, a reviewer suggested replacing a ternary expression by an `if` statement to improve understandability. These two code reviewers suggest the opposite operations, both concerning understandability. This means they perceive understandability improvements (in the context of the usage of `if` statements vs. ternary expression) in a different way.

Style guides (a.k.a. code conventions) also are adopted to improve code understandability in the real world. Smith et al. [23] defined that "*code conventions are a body of advice on lexical and syntactic aspects of code, aiming to standardize low-level code design under the assumption that such a systematic approach will make code easier to read, understand, and maintain*". However, the recommendations proposed by different style guides are not only about understandability and can also be divergent. For example, while the Google Java Style Guide³ suggests the use of braces even when the block is empty or contains only a single statement (e.g., `if (a) {b+c}`), the Linux Kernel Coding Style⁴ disagrees with it for single statements (e.g., `if (a) b+c`). This suggests that some code alternatives that make source code more under-

3. <https://google.github.io/styleguide/javaguide.html>

4. <https://www.kernel.org/doc/html/v4.10/process/coding-style.html>

standable are, to some extent, in the eye of the beholder, e.g., they depend on the experience and context of the developer. However, there is insufficient empirical evidence of which code alternatives are preferred during software development considering code understandability to support the creation of a style guide. In this study, we investigate those code alternatives to further help developers choose the best practices and tool builders to create automated support based on empirical evidence from software development teams. We do so in the context of code review, where developers are quality gatekeepers who review and discuss changes until they consider the code is good to be merged into their projects.

3 STUDY DESIGN

This section presents the research questions we aim to answer (Section 3.1), the data collected for the study (Section 3.2), and the process for identifying code review comments that suggest understandability improvements (Section 3.3).

3.1 Research Questions

RQ1. *How often do reviewers ask for code understandability improvements in code review comments?*

Motivation: While code reviews are commonly employed to enhance code quality [11], [12], there is a lack of information regarding the prevalence of code review comments focusing on code understandability. If a substantial portion of code review comments involves understandability improvements, it may be advantageous to prioritize automation strategies tailored to address this specific area. Furthermore, not every improvement targets executable source code and it may be useful to quantify the extent to which other scopes, such as code comments and string literals, are targeted. Such targeted approaches could lead to significant time and effort savings.

Method: We analyzed a representative sample of code review comments to determine the rate at which reviewers suggest code understandability improvements. In this direction, we manually classified 2,401 code review comments selected as explained in Section 3.2, assessing whether each comment pertains to improving code understandability. Furthermore, we categorized the associated scopes of these comments (e.g., documentation, executable source code, string literals). Each code review comment was classified by two authors of this paper. We calculated the Kappa coefficient [24] to assess the agreement between their evaluation. We found $k = 0.59$, which is considered a moderate agreement strength [25]. For every case where there was a disagreement, three or more authors engaged in discussion to reach a final classification. Besides this classification, we also verify the scope to which each comment refers, i.e., where in the source file the reviewer has identified an opportunity to improve understandability (e.g., source code and Javadoc). Additionally, we randomly selected and classified 385 code review comments related to code understandability. We distinguished between explicit comments, which indicate that the improvement suggestion is motivated by code understandability, and implicit comments, where the intention

of code understandability improvement is inferred from the comment text, code snippet, and the code improvement itself. We also provided a description of why we considered each code review comment to be related to code understandability. In the sample, we found a total of 50 explicit comments and 335 implicit comments related to code understandability.

Novelty: Previous studies [26], [27], [28] explored the projects' files histories to identify code understandability improvements based on state-of-the-art metrics or commit messages that reported an understandability improvement explicitly. We evaluate 2,401 code review comments and identify suggestions by code quality specialists (reviewers) to improve source code understandability. Unlike previous work [29], we also examine when the intention to improve code understandability is implicit.

RQ2. *What are the main issues pertaining to code understandability in code review?*

Motivation: Identifying the typical understandability issues reported by reviewers can guide researchers and tool builders when designing and training machine learning-based tools to improve understandability. Additionally, this knowledge can contribute to the development of code review checklists [30] to make the process more systematic and efficient to address code understandability concerns in a structured manner [31].

Method: We selected 385 code review comments randomly from the 1,012 reviews that were related to code understandability improvements and were classified in Section 4.1. This sample size of code review comments is a representative sample with a confidence level of 95% and a margin of error of 5% of the unlimited population, according to Cochran's formula [32]. We then identified the issues that triggered the suggestions for change (the what) and grouped them into categories that we called *understandability smells*. For example, in one code review [key=821], the reviewer asked to remove three code elements because they were constants that were not being used, i.e., UNNECESSARY CODE (the "what"). Also, we identified the type of code element targeted for improvement (the "where") using Spoon [33], a well-established open-source library for analyzing Java source code. Three authors examined the sample of 385 code review comments. They described the issue presented in each comment and identified where the issue occurred in the code. The same authors then discussed the issues and grouped similar comments together. Afterwards, the groups were refined, and categories were established during meetings involving at least five authors.

We used the nodes of Spoon's AST to group constructs related to each issue pointed out by code reviewers into categories. For example, we group the nodes `CtIf` and `CtSwitch` into the Conditional category⁵. We considered the most specific AST node to which an understandability smell is associated to determine where it happens. For example, in a code review [key=123], a reviewer requested using an alternative method invocation to evaluate an expression

5. The `CtIf` node refers to an 'if' statement in the AST, while the `CtSwitch` node refers to a 'switch' statement. Both are conditional statements in the Java language.

in an `if` statement, and we categorize the most specific node identified by `Spoon`, `CtInvocation`, as a `Call`. However, in other scenarios, the understandability smell consists of a set of nodes, and we considered the node that is the closest parent of all other nodes. For instance, in another code review [key=1878], the reviewer asked to rewrite part of the method's body, and the AST was composed by several nodes such as an invocation and an `if` statement. In this case, the closest parent of those nodes is a `Method`, represented by `CtMethod` in `Spoon`. We also categorized understandability smells as `Method` when the reviewer requested renaming the method, adding Javadoc to it, or moving it. In two code review comments where the reviewers pointed out multiple issues in different parts of the source code, we classified the "what" and "where" exclusively based on the location where the code review comment was marked. For example, in a code review comment in the class declaration where the reviewer asked to "remove the `Db` [from class name] and name attributes from the annotation" [key=1772], the code review comment is at the line of the class declaration and we considered that this (Class) is the place where the issue was pointed out.

Novelty: Previous studies [26], [27], [28] classified the issues related to code understandability in terms of software maintenance types and rules of static analysis tools. However, their classification is limited to these terms, which do not capture the actual code understandability issues. In this work we examined code review comments, as in the work of Dantas et al. [29], but take two complementary perspectives: coarser-grained, by eliciting the high-level issues that trigger improvements, and finer-grained, by examining which source code elements are connected to these issues.

RQ3. *How likely are understandability improvement comments to be accepted?*

Motivation: Analyzing the odds of developers accepting comments when they suggest understandability improvements highlights the importance or lack thereof of this kind of comment. It might provide evidence of cases where new tools could be created or not for automatically addressing the identified problems.

Method: We aimed to investigate if recommendations for improving code understandability are more likely to be accepted than code reviews that do not address code understandability. To achieve this, we took the pool of 385 code reviews that were analyzed for RQ2 and added another random sample of 385 code reviews that suggest changes unrelated to code understandability. The sample was divided among the three first authors. In the first step, the authors searched for revisions in the part of the code where the reviewer left their comments, looking at each new version of the file in the pull request. Eventually, it was necessary to look at other parts of the code that were indicated by the review comments. In the second step, the authors determined whether the reviewer comments were addressed (accepted) by the developers in any of those revisions. We classified improvement suggestions in reviews as either accepted or not accepted based on whether or not we find the code changes as suggested by the reviewer. We calculated the odds ratio to determine the likelihood of

accepting a suggestion, given it is a code understandability improvement.

Novelty: Brown and Parnin [34] investigated the acceptability of *GitHub suggested changes*, a special kind of code review comment where reviewers recommend a specific change to the code line or lines. However, as far as we know, no study evaluated how developers addressed reviewer's suggestions for code understandability improvements.

RQ4. *What code changes are found in understandability improvements?*

Motivation: There are multiple studies [7], [8], [16], [17], [18], [19], [35] that empirically evaluated the impact of different ways of writing code on understandability in a lab setting. However, understandability strongly depends on the context where the code must be understood, e.g., due to the experience of the readers. This RQ aims to help developers and researchers to know what makes the code more understandable in practice. Information about typical real-world understandability improvements can help us establish what kinds of tools can address each scenario (linters, more in-depth static analysis, LLM-based, etc.).

Method: We took as a starting point the set of code reviews examined for RQ2 and RQ3. For this question, we focused on the code review comments where the recommendation made by the reviewer was accepted, and a patch was applied as a consequence. We analyzed the code change applied in each patch to resolve the problem raised by the reviewer. The sample was divided among the three primary authors, who described the improvements made in response to the reviewer's comments. We then organized these different types of changes into categories based on the understandability smells identified in RQ2. Finally, we discussed how the improvements could be organized in meetings with at least four of the authors.

Novelty: As with RQ2, we adopted a bottom-up approach where we identify code understandability improvements in terms of the fine-grained refactorings that developers perform. Previous studies [26], [27] employed a top-down approach, classifying the understandability smells in terms of software maintenance types and rules of static analysis tools.

RQ5. *To what extent are accepted code understandability improvements reverted?*

Motivation: Understanding the frequency at which code understandability improvement patches are reverted after being applied during the code review process is essential for assessing the subjectivity of code understandability improvements. Frequent reversions of such patches may indicate a high level of ambiguity and variability in the perception of code understandability, which can lead to challenges when utilizing this data to train machine learning systems, potentially resulting in suboptimal performance [36].

Method: For this RQ, we employed the same set of code review comments that were analyzed for RQ4, i.e., comments about understandability where the recommendation made by the reviewer was accepted and a patch was applied. We manually searched subsequent versions of the files modified in that patch to determine whether the patch had been

reverted. We considered that a reversion happens when (i) a reviewer suggests improving code understandability, (ii) that suggestion is accepted, and a patch is submitted as a consequence, incorporating said suggestion, and (iii) subsequently, a new patch is submitted, undoing the understandability improvement. In this process, we account for renamed files and cases where portions that were affected by the patch were moved to other parts of the system, e.g., a different file. If the latest version of the file still includes the patch that was introduced as a response to the suggestion for improvement, we considered that it was not reverted. If the file as a whole or the element (method, class, etc.) where the modification was performed is deleted, we also considered that the change was not reverted since we cannot ascertain whether the deletion is connected to said change.

Novelty: Kalliamvakou et al. [37] explored the integration rate of proposed changes in GitHub pull requests into the codebase, while Shimagaki et al. [38] assessed the frequency of commits that were subsequently reverted in project histories. However, a gap remains in the existing literature, as no study specifically examines the adoption and reversion of code understandability improvements within the context of code reviews and the history of the codebase.

RQ6. *Do linters contain rules to detect the identified code understandability smells?*

Motivation: Linters are widely used in real-world software development [39], [40], [41]. Furthermore, they are capable of detecting several typical problems in source code. It is then worth investigating the extent to which they capture the code understandability issues that code reviewers point out. Ideally, reviewers should not have to worry about problems that an automated tool can capture.

Method: We investigated the coverage of the code understandability smells identified in our study by rules implemented in Spotbugs [42], PMD [43], SonarQube [44], and Checkstyle [45]. These four tools have the highest number of rules [41], are widely used by practitioners [40], and have been studied by researchers [46]. Together, they have a total of 1,315 rules. We manually checked each observation of an understandability smell against each tool to determine if it could detect it.

We utilized the set of code review comments that were previously analyzed for RQ4, which are about understandability smell instances. For each understandability smell, we manually inspected the descriptions of the rules implemented in linters to find one that could detect the smell. In the end, we calculated the coverage of understandability smells by computing the number of instances that could be detected by at least one of the linters.

Novelty: Previous work [29] investigated the extent to which SonarQube can detect code understandability issues in code reviews by running this tool. In contrast, we manually analyzed the applicability of every rule implemented by four different linters. The issues reported by linters depend on how they are configured [47], as linters sometimes have contradicting rules, e.g., SonarQube includes a rule *Close curly brace and the next "else" [...] should be on two different lines* but also one that ends with *...on the same line*. A manual analysis can capture these inconsistencies.

3.2 Data Collection

To answer our research questions, we selected open-source projects with a code review process active in GitHub, i.e., projects where new pull requests are created frequently. Our focus is on the code review comments made by project contributors regarding the code changes submitted through these pull requests. Furthermore, those projects must be non-trivial software systems, e.g., applications, infrastructure components, libraries, frameworks, with many contributors.

We started out by using SEART [48] to search for project candidates because it offers appropriate search criteria. In SEART, we considered the following criteria: (i) Java projects, (ii) at least ten stars (fixed by SEART), (iii) at least one pull request, (iv) at least ten contributors, and (v) not a fork repository. We selected projects with ten or more contributors because we need a number of contributors that justifies the code review process. These criteria contribute to filtering out personal projects [37]. SEART returned a list of 9,735 projects.

Next, we filtered the projects to select only those where code changes submitted by developers are regularly reviewed by other contributors. Thus, we selected projects with at least one pull request per month in 2020 that include one or more inline code review comments. These requirements ensure that code reviews are commonplace and happen at the level of source code lines. Given that our search was conducted in October 2021, we focused on pull requests submitted during the year 2020. With this filter, we kept 389 projects.

We further refined this set by removing projects where we can not identify the pull requests that were merged since merging a pull request is a clear criterion to determine whether said pull request was accepted. Merging can be done manually through *git* commands on the local repository and then pushing to the remote. Alternatively, it can be done by using the merge button directly on the GitHub platform. Pull requests with automated merging have a clear indication of this event on the pull request's timeline, with its status updated to "merged." However, identifying projects with manual merging can be challenging as it requires the use of heuristics to determine the commit that merged the pull request, as explained by Kalliamvakou et al. [37]. We identified eight projects (e.g., apache/gobblin and openjdk/skara) where the pull requests are not merged automatically and decided to remove them. After performing these filtering steps, we kept 381 projects with an active code review process that integrates their code changes in their main branch through GitHub.

Then, we performed a fine-grained filtering of these 381 projects. We manually analyzed each one and evaluated whether it is mainly composed of code. As a result, we excluded eight projects that do not fit our software criteria, such as those classified as documentation. Also, we constrained our investigation to projects where most code review comments are written in English. We removed seven projects where most code review comments are not in English. In addition, one project (returntocorp/semgrep) did not primarily use Java, and another (apollographql/apollo-android) was transitioning from Java to Kotlin. As a result,

TABLE 1: Descriptive statistics for the code review comments, discussions, and source code lines associated to these comments and discussions, for the sample of code reviews in the 363 selected projects.

	Mean	Std	Min	25%	50%	75%	Max
# comments/project	6.6	2.3	1	5	7	8	15
# chars/comment	120.2	216.9	0	30	72	148	44,043
# chars/line of code	53.0	95.9	0	30	49	72	22,382
# chars/discussion	224.2	435.7	0	50	115	247	45,101
# comments/discussion	1.9	1.2	1	1	2	2	43

we removed both. Finally, the code review comments in one project (fisco-bcos/web3sdk) are all made by bots, such as “sonarcloud”. This project was also removed, leaving us with a final total of 363 projects⁶.

The selected 363 projects comprise more than 349,000 code review comments made in 2020. We downloaded these code review comments using the GitHub GraphQL API⁷. Since we need to manually evaluate and classify the code review comments, we extracted a statistically representative sample. We chose a confidence level of 95% and a margin of error of 2%, resulting in a sample size of 2,401 code review comments for an unlimited population size. We employed a stratified random sampling [49] strategy, as the resulting sample can be more representative of the investigated population of code review comments than if we just employed simple random sampling [50, Chapter 11], where projects with few code review comments might not be represented. We randomly selected code review comments from the projects in proportion to the total number of code review comments each project has. As a result, the minimum number of code review comments per project is 1, the maximum is 15, and the median is 7. The complete list of analyzed comments can be browsed at <https://codeupcrc.github.io>. Table 1 presents descriptive statistics of these code review comments and their associated discussions. The table indicates that, in some cases, code review comments can be zero characters. These comments comprise only images (Figure 3 presents an example), bullet points without accompanying text, emojis, or are simply empty.

3.3 Identifying Understandability Improvements

To answer the research questions of this study, we first classified whether code review comments suggest understandability improvements or not. In this classification process, we need to identify the intention of the reviewer based on their comment. Sometimes, this intention is clear and explicit in the comment text. For example, in Figure 5, the reviewer asked to rename a local variable because “it will help code more readable down into method [...]”. However, other comments are terse, and the intent is not explicitly stated. For example, in Figure 6, the reviewer asks to rename *deadBrokers* to *deadBrokersWithUnknownCapacity* but the intention is not explicit. Although the reviewer does not write it, the most likely motivation is that the new name helps developers better understand the purpose of this variable.

When the inline comment is not enough, it is necessary to consider more information in the context, i.e., the replies

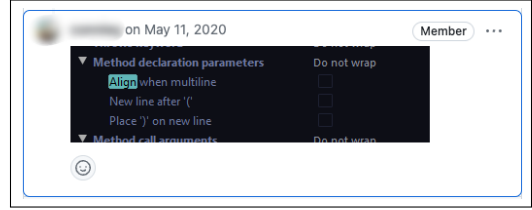


Fig. 3: Example of comment comprised solely of images [51].

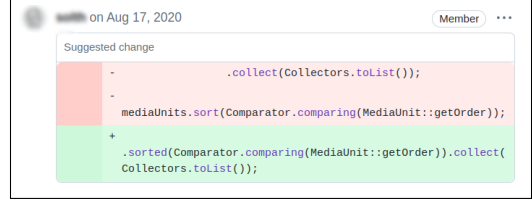


Fig. 4: Example of a suggested change to preserve the function but change the logic [52].

to the comment, the source code, and the patch (when available). We first checked whether the suggestion does not change the functionality of the code. For example, in Figure 7, the reviewer asks the author of the pull request to pass another object to the function *contains* instead of the variable *col*. We first evaluated whether the suggestion changes the function of the code snippet to which the suggestion refers because it could indicate that the reviewer is suggesting a correction or different functionality. When we look at the patch, we confirm that the suggestion is behavior-preserving, i.e., the old and the new versions produce the same result given the same input. Thus we consider it as an understandability improvement. However, we found some comments where the logic of the source code changed, but the function is the same. For instance, in Figure 4, the reviewer suggests changing how a list is processed in the stream, but the expected result of the stream is preserved. We also consider these cases to be potential understandability improvements.

In summary, we classified a code review comment as suggesting an understandability improvement if it fits within one of the following cases: (i) the intent to improve understandability is explicit in the text, (ii) it is a comment about aspects that do not affect the compilation or the execution of the program, i.e., code comments or code formatting, (iii) the refactoring that it suggests is motivated mainly by improving understandability, as discussed by Fowler [53], or (iv) it does not explicitly state its intent, but it makes a suggestion that is very similar to a comment where the intent is explicit, i.e., the suggestions for improvement require the same change, e.g., in an explicit code review comment, the type *Boolean* is replaced by *boolean* to ensure consistency [key=1201], similarly, in an implicit code review comment, the type *LinkedList* is replaced by *ArrayList* [key=1089] for the same reason. In a code review comment, the reviewer could suggest an improvement that impacts multiple factors, such as performance and understandability. We also consider those types of improvements that impact understandability. Each comment, including those where the reviewer explicitly mentioned that it pertains to understandability or those that are naturally about understandability (as described in Section 2.2), was classified

⁶. The list of selected projects can be found at <https://codeupcrc.github.io/projects.html>

⁷. <https://docs.github.com/en/graphql>

by at least two authors. Conflicts between answers were resolved through meetings involving at least five authors. In the meetings, we analyzed the conflicts through discussion sessions and resolutions were achieved by consensus. When consensus could not be achieved, the code review comment was discarded and a new one was selected for labeling. In the remainder of this section, we explain the types of comments that required additional analysis.

3.3.0.1 Ambiguous intention comments: There are scenarios where it is impossible to discern the intention of the reviewer. For example, in a comment [54], the reviewer suggested “better to use `putString`”. This suggestion can be related to improving code understandability or improving performance. There is no evidence of any intention, neither in replies nor patches. If we cannot establish intention with some confidence, we discard the comment.

3.3.0.2 Repetition comments: Similarly, in other code review comments, we find tiny comments, e.g., “ditto” [55] and “same here” [56], that do not bring any information about the reviewer’s intention. It would be necessary to look at another code review comment to get the context of the comment, and looking for the original comment could be time-consuming. Since these comments account for approximately 1% of our sample, we decided to disregard them. However, we kept them in the dataset labeled as “Discard” for future research.

3.3.0.3 Question comments: Sometimes reviewers use the code review comments to ask questions to the PR author. On the one hand, there are clarifying questions where reviewers attempt to understand the purpose of the code snippet under review (e.g., “Why is this needed?” [57], “Why protected?” [58]). In those scenarios, we considered that the comment does not suggest an understandability improvement. On the other hand, we found scenarios where reviewers suggested understandability improvements through questions. For example, in a code review comment, the reviewer asked “wrong indent?” [59]. This suggestion, phrased as a question, indicates a problem with how the code is formatted. This kind of code review question is considered related to understandability improvements. Previous work [60] has shown that comments with different purposes are often phrased as questions.

3.3.0.4 Multi-purpose comments: In other code reviews, the reviewers suggested changes with multiple purposes. In one example [61], the reviewer commented: “[...] Suggested change `if (path.contains("b/")) {` → `if (path.startsWith("b/")) {` for better performance [...] and preferably change the name of the `urlString` variable back to just `url`” [key=2151⁸]. Although the reviewer suggested a change to improve performance, they also asked to rename an identifier intending to improve the code understandability. This kind of code review comment was considered related to understandability improvements. In our understanding, it is a single comment suggesting multiple different changes.

3.3.0.5 Bot comments: Finally, we found code review comments introduced by bots. Some of them are sug-

8. Hereafter, we use the reference [key=####] to represent a link [https://codeupercr.github.io?key=####], pointing to the code review comments in our dataset.

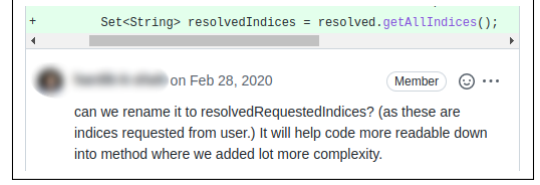


Fig. 5: The reviewer explicitly asks for understandability improvement [63].

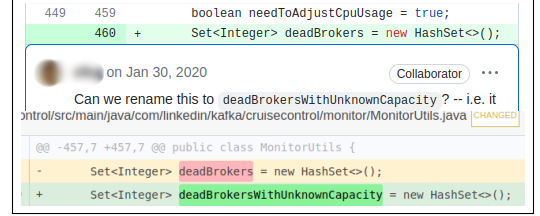


Fig. 6: The reviewer implicitly asks for understandability improvement [64].

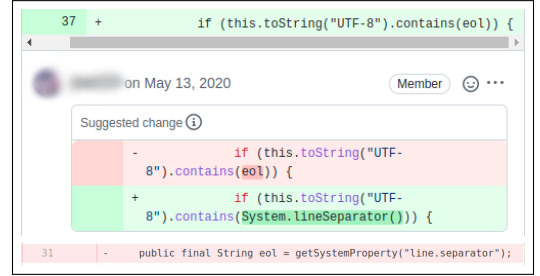


Fig. 7: The intention of the comment is ambiguous [65].

gestions based on linters to improve understandability. For example, the bot named *codeclimate* created the following code review comment [62]: “Method `buildMenu` has 59 lines of code (exceeds 30 allowed). Consider refactoring”. Although the suggestion of the code review comment is related to understandability improvement, we discarded these code review comments because there was not an evaluation by a human specialist (i.e., a reviewer). Previous work [39] has shown that many recommendations made by linter bots are ignored by reviewers.

4 RESULTS

4.1 How often do reviewers ask for code understandability improvements in code review comments? (RQ1)

Table 2 presents the results of our manual classification of code review comments in terms of whether they pertain to code understandability or not. Each row indicates the number of comments per scope, which can be related to (i) executable source code (CODE), i.e., excluding comments and blank lines and focusing on the instructions that will actually be compiled and executed, (ii) high-level API documentation (JAVADOC), (iii) regular source code comments (COMMENT), (iv) a string value or expression with strings (STRING LITERAL), or (v) OTHERS (e.g., annotations). We separated the comments to generate documentation and regular comments because they have different characteristics. JAVADOC is well structured, has special annotations to reference specific elements in the documentation, and is used to explain classes, attributes, and methods, often for readers who are not looking at the code. COMMENT has

no prescribed structure and is used to explain the code or the rationale behind it in a more direct manner for readers who are directly looking at a specific part of the code. Also, we analyzed STRING LITERALS separately because these literals may give developers hints about program behavior, e.g., in log messages, but they are not comments. Finally, annotations, log levels, and other special cases are analyzed separately. We did not consider annotations to be part of the executable source code because, according to the Java Language Specification⁹, they have “no effect at run time”.

In the analyzed sample, we found out that 1,012 (42.15%) of the code review comments include suggestions, requests, or discussions about code understandability improvements. This amounts to more than two out of every five code reviews. This result emphasizes that reviewers are worried not only about the correctness and performance of the code but also whether the developers are following the best practices of programming and code understandability. We check whether comments suggesting code understandability improvements differ from other comments by comparing their sizes. If one kind of comment tends to be significantly shorter, this may suggest that developers are willing to dedicate less time and effort to it. The 1,012 comments suggesting code understandability improvements tend to be shorter (99.21 characters) than those not related to understandability (117.70 characters). The difference is statistically significant (Mann-Whitney U, $p = 0.000001$) but the effect size is negligible (Cliff’s Delta, -0.12). As for the associated source code lines, i.e., the lines where the inline code reviews were added, the mean character counts are 50.52 and 54.15 and the difference is not statistically significant (Mann-Whitney U, $p = 0.07$). Overall, we can say that the difference between comments and source lines of code associated with understandability improvements and those that are not is inconsequential.

Considering all code review comments analyzed (2,401), the majority of them (82.2%, 1,974 out of 2,401) are in the scope of CODE. Among the 1,974 code review comments whose scope is CODE, 33.49% (661 out of 1,974) are related to code understandability improvements. The reviewers in these code review comments have asked code submitters to rename identifiers, remove unnecessary code elements, use alternative code constructs, and others. We discuss the content of those code review comments in Section 4.2. The results support other studies [11], [12] that indicate that improving code quality is one of the main goals of code reviews.

When we look only at the code review comments in the scope of JAVADOC, 86.26% (157 out of 182) of them are about understandability improvements. Generally, reviewers ask code change submitters to add Javadoc documentation (e.g., “Missing javadoc” [key=747]), improve it (e.g., “@link on this and the next line please” [key=802]), or fix the description (e.g., “appear → appears” [key=819]). We found examples of code review comments related to Javadoc that do not suggest understandability improvements. For example, the code review comment “what if these objects don’t have a common ID or don’t have a common event time?” [key=26] discusses the design solution described on the Javadoc. In this case,

TABLE 2: Classification of the 2,401 code review comments in terms of whether they pertain to code understandability or not. The two groups are further divided based on the scopes to which the code review comments refer. The percentages for each row add up to 100%.

Scope	Is it about understandability?		
	Yes	No	Discarded
CODE (1,974)	33.49% (661)	63.93% (1,262)	2.58% (51)
JAVADOC (182)	86.26% (157)	12.09% (22)	1.65% (3)
COMMENTS (110)	78.18% (86)	20.91% (23)	0.91% (1)
STRING LITERAL (81)	80.25% (65)	18.52% (15)	1.23% (1)
OTHER (54)	79.63% (43)	20.37% (11)	0% (0)
Total (2,401)	42.15% (1,012)	55.52% (1,333)	2.33% (56)

the documentation is not the target of the discussion but the code design related to this documentation.

Similarly to JAVADOC, 78.18% (86 out of 110) of the code review comments in the scope of COMMENTS are related to understandability improvements. The reviewers suggest the addition (e.g., “Please add comments where you see fit.” [key=1287]), improvement (e.g., “Nit: Typo shake → sake” [key=182]), or removal (e.g., “we should remove this comment” [key=1062]) of comments in the source code. Conversely, some of the code review comments that impact comments do not relate to understandability improvements. They are related to comments with tags such as TODO and FIXME.

In the analyzed code review comments related to STRING LITERAL scope, 80.25% (65 out of 81) can be considered code understandability improvements. In those code reviews, the reviewers suggest changes to string literals to improve exception messages (e.g., “Can you add what the expected and actual partition path was in the exception message ?” [key=177]), assertion error messages (e.g., “Can you please change this to be cookieMaxAge cannot be zero?” [key=2096]), logging messages (e.g., “I suggest that you can change to `logger.error("unknown error", e);`” [key=2294]), and other method arguments (e.g., “The number of threads...ingest → This property is deprecated since 2.1.0. The number of threads...” [key=60]). In contrast, in the code review comments unrelated to understandability improvements, the string literals are arguments used to send messages to the end user (e.g., “Let’s change the description to “List of transports the GAPIC can use...”” [key=1008]), the paths of a resource on disk (e.g., “I think you have a typo in the filename: the file is: ClangAnalyzer.txt the fileName is: ClangAnalyzer.txt” [key=1259]), and tags used on the header of a protocol (e.g., “TFB → ServiceTalk...” in `.addHeader("Server", "TFB")` [key=2157]).

In the OTHER scope, the code review comments are related to annotations and logging (excluding comments that directly refer to string literals). From those, 79.63% (43 out of 54) are related to understandability improvements. The reviewers ask code change submitters to change (including adding and removing) logs and their configuration (e.g., “Do we need this log here or was it just for debugging?” [key=2372], “Use `log.info?`” [key=1896]) and add documentation annotations (e.g., “@PublicAPI instead of @beta” [key=839]). We also found code review comments that not support understanding the source code where reviewers ask for modifications in annotations that may be related to bugs (“Does the lazy inject actually work since `AuditEventLoggingFacade` is not a lazy bean?”

9. <https://docs.oracle.com/javase/specs/jls/se12/html/index.html>

[key=1545]) or modifications that are related to release notes (“An entry in *CHANGES_NEXT_RELEASE* describing the fix...should be included” [key=2161]), for example.

Answer to RQ1. We found that 42.15% (1,012/2,401) of the selected discussions are related to code understandability improvements. Among them, 65.31% (661/1,012) are about executable source code, almost 24.01% (157+86 out of 1,012) are related to comments, and more than 10.67% (65+43 out of 1,012) are related to literal strings, annotations, and others. Approximately one out of every three code review comments that refer to executable source code is about understandability. Furthermore, four out of five code review comments that refer to elements other than executable code in a program source file pertain to understandability improvements.

Implications. In previous research [27], [28], the researchers identified ways to improve code understandability at the level of files by analyzing commit messages. Our findings suggest that researchers and developers can also learn from code reviews to improve code understandability at the level of source code snippets. Moreover, our results demonstrate that automated tools that enhance code understandability can have a significant impact in reducing developers’ effort during code reviews. Additionally, our work makes it clear that improving code understandability requires tools that work with both programming language and natural language elements.

4.2 What are the main issues pertaining to code understandability in code review? (RQ2)

Table 3 shows the frequency of the understandability smell (rows) found in our sample and where they are in the source code (columns), with each cell matching the understandability smell in a specific place in the source code. We try to keep the names of the understandability smells as self-explanatory as possible. Therefore, we sometimes employ longish names such as “*Incomplete or inadequate code documentation*”, instead of more typical smell names such as “*Shotgun surgery*” and “*Comments*” [53]. We categorized eight kinds of understandability smells found in 16 places in the source code. More specifically, we encountered the following places (with the number of occurrences in parentheses): Method (130), Call (55), Literal (40), Class (36), Attribute (29), Conditional (27), Variable (22), Parameter (11), Import (8), Operation (7), Try-Catch (7), Loop (5), Annotation (5), Interface (1), Object (1), Package (1). The categories with less than 10 instances of understandability smells were grouped under Others in the table.

The most prevalent understandability smell is INCOMPLETE OR INADEQUATE CODE DOCUMENTATION, which appears in 22.3% (86 out of 385) of the code review comments. It groups the cases where the concerns are in documentation and code comments. There are comments where reviewers point out concerns related to Javadoc, inline code comments, and Java annotations. In general, the reviewers indicated missing documentation (e.g., “*Add a comment indicating whether start/end are inclusive or exclusive*” [key=2020]); incorrect documentation (e.g., “*Minor: which is available...*” [key=1115]); incorrect placement of documentation (e.g., “*Think you may have intended to have the below comment here too, or only here since it’s the first occurrence*” [key=1724]); and

unnecessary comments (e.g., “*What does the comment mean? Or can it be removed?*” [key=321]). Furthermore, the reviewers indicated incorrect or missing documentation about the status of methods using the Java annotation `@Deprecated`. More than half ($45/86 = 52.3\%$) of the occurrences of this understandability smell are related to Method (39 in declaration clause and six in body block) and about 17.4% (15/86) are related to Class. We find more understandability smells about code documentation in these places because, in Java, code documentation is frequently written in methods and classes.

The second most popular understandability smell is BAD IDENTIFIER, which appears in 20.3% (78 out of 385) of the code review comments. The reviewers pointed out concerns related to the content of the identifier name and its style. A good identifier can help better communicate the content or its functionality, e.g., the variable `n` that represents the person’s name is better represented by identifier `name`. We highlight code review comments indicating typos in the identifier names (e.g., “*Typo in variable name*” [key=811]); parameter names in upper case (e.g., “*Would ... making ... the MBI parameter in createPatientResource() lowercase? Might get mistaken for constants...*” [key=579]); method names that do not explain functionality (e.g., “*This method actually returns the Requirements class, not the annotation, so better call it getArtifactRequirements()*” [key=535]); and variable names incompatible with their type (e.g., “*Rename the field as well? Since it’s now an executor and not a handler...*” [key=340]). Most of the occurrences of this kind of understandability smell are in Method (32 in the declaration clause and two in the body block), Attribute (14), Variable (11), and Class (10).

Another understandability smell is COMPLEX, LONG, OR INADEQUATE LOGIC. The reviewers pointed out concerns about complex and long alternatives to writing expressions, code constructs, and other statements. For instance, the boolean expression `map.keySet().size() > 0` could be replaced by a unique method invocation, `!map.isEmpty()`. We find this category in 18.2% (70/385) of the observations in our sample. The reviewers have identified redundant method calls (e.g., “*Looks like it could be simplified*” [key=83]); and long method implementation (e.g., “*Maybe you can refactor this logic into a different method, it makes this method quite large...*” [key=1413]). We also found comments suggesting that anonymous inner classes are too verbose (e.g., “*With a lambda, this would be slightly shorter*” [key=58]); and complex if-else implementation with repeated code (“*I think a better way to write this method is: [duplicate code within if-else out of if-else statement]*” [key=1878]). Moreover, reviewers indicated concerns related to using API classes, such as using a class as type instead of its primitive type (e.g., “*Why Boolean and not boolean?*” [key=1201]). This smell is more frequent in Call (27), Methods (eleven in the body and two in the declaration clause), and Conditionals (nine).

UNNECESSARY CODE occurs in 13.2% (51 out of 385) of the code review comments in our sample. This understandability smell involves snippets of source code that can be safely removed without altering the functionality of the code. This goes beyond simpler scenarios, such as textually duplicated or dead code. For example, in the code snippet `V1=2; V1+=1; V1=1`, the two first state-

TABLE 3: The frequency of understandability smells found by reviewers and where they are in the source code. For the code elements marked with an asterisk, the understandability smells were found in their declaration, access, or implementation.

Understandability smells	Method*	Call	Literal	Class	Attribute*	Conditional	Variable*	Parameter*	Others	Total
INCOMPLETE OR INADEQUATE CODE DOCUMENTATION	45	4	1	15	3	9	3	0	6	86
BAD IDENTIFIER	34	0	0	10	14	0	11	9	0	78
COMPLEX, LONG, OR INADEQUATE LOGIC	13	27	0	2	3	9	4	1	11	70
UNNECESSARY CODE	17	8	1	2	8	3	3	0	9	51
INCONSISTENT OR DISRUPTED FORMATTING	17	4	1	7	1	6	0	0	6	42
WRONG, MISSING, OR INADEQUATE STRING EXPRESSION OR LITERAL	0	1	29	0	0	0	0	0	1	31
INADEQUATE LOGGING AND MONITORING	3	11	1	0	0	0	1	0	2	18
MISSING CONSTANT USAGE	1	0	7	0	0	0	0	1	0	9
Total	130	55	40	36	29	27	22	11	35	385

ments do not influence the final value of `v1` but may still impose a cognitive load on the reader and, in the worst case, may cause her to misjudge the final value of `v1`. Among other elements, the reviewers identified unused imports, classes, methods, variables, and constants (e.g., “unused constant” [key=816]); and commented out source code (e.g., “commented code” [key=1074]). Other concerns are related to redundant code, such as unnecessary use of `this` (e.g., “nit: this is redundant here” [key=1871]); unnecessary intermediate variable (e.g., “Could be better to have the full `clientSession.newRequest().execute()` to avoid having `loadCsv` variable” [key=1561]); and semantically duplicate method (e.g., “Is this necessary given that `super.getDependencies` does the same?” [key=1741]). We encounter this smell more often in Method (17 – twelve in the declaration clause and five in the body block), Call (8), and Attribute (8). These three comprise more than half of the instances in this category.

In 10.9% (42/385) of the code review comments the reviewers identified INCONSISTENT OR DISRUPTED FORMATTING. This understandability smell is linked to spaces, braces, parentheses, and formatting styles. The formatting elements can help the reader to identify and delimit the elements in source code, e.g., the use of curly braces (`{}`) to define the scope of a block. This includes comments that identified missing vertical and horizontal space (e.g., “more spaces please” [key=683]); missing parentheses to make expression evaluation order explicit (e.g., “Suggested change `...requiresMaintenance() && null != u.getTech()` → `...requiresMaintenance() && (null != u.getTech())`” [key=1453]); and missing brackets and line breaks. The reviewers also pointed out extraneous vertical and horizontal spaces and parentheses (e.g., “A little too many parens here” [key=1446]). Additionally, we run into comments that indicate code formatting that does not match formatting guidelines, e.g., in `if` statements, “IF statement should be on it’s own line” [key=2024]). These cases of inadequate formatting are most frequently 57.1% (24/42) found in Method (17 – seven in the declaration clause and ten in the body) and Class (7).

We find in 8.1% (31/385) of our sample code review comments related to WRONG, MISSING, OR INADEQUATE STRING EXPRESSION OR LITERAL. This understandability smell pertains to concerns regarding natural language string messages such as those used as arguments in exceptions or logs, as well as scenarios involving the style and typographical correctness of string values. In these comments, the reviewers indicated incorrect words in

string messages (e.g., “... iterable, not iterator...” [key=2365]); different styles of string message (e.g., “not supported sounds a bit better than “unsupported”” [key=2287]); and missing string messages (e.g., “Maybe an explanation here would be nice?” [key=1345]). Moreover, the reviewers pointed out the string value in a parameter that does not match the code conventions (e.g., “Suggested change `@Column(name = "END_TEXT")` → `@Column(name = "end_text")`” [key=1418]); and a typo in a string value (“Suggested change `operationId = "aproveInboxItemById"` → `operationId = "approveInboxItemById"`” [key=1642]). By definition, all comments where this smell manifests pertain to concerns found in existing literals during code review. There are two exceptions to this pattern: when the smell is found in a throw invocation without a literal, and when the format method is used to construct a string. In the first case, we identify the Try-Catch node as the source of the smell, and in the second case, we consider the Call node.

The instances of the INADEQUATE LOGGING AND MONITORING smell account for 4.7% (18/385) of the code review comments. This comprises scenarios related to the logs and exceptions statements. These statements communicate to developers information about system behavior at run time, e.g., for debugging and monitoring purposes. This information can also help developers to make sense of the code functionality around these statements. In these comments, reviewers indicated concerns about the usage of logs and exceptions, such as missing log (“should we log this? just to make investigation easier” [key=1135]); unnecessary log (“In my opinion, we do not need this log. WDYT?” [key=2337]); incoherent log levels (e.g., “...this is currently logging every second in test. Can this be changed to debug?” [key=574]); and throwing exceptions of generic types (e.g., “This is not a very good exception, let’s tell the developer what is wrong in each case and how they could fix it” [key=919]). Most of the instances of this understandability smell are related to Call (11/18) statements. This smell does not cover issues with text literals used as log messages, as they are instances of the WRONG, MISSING, OR INADEQUATE STRING EXPRESSION OR LITERAL smell.

Finally, 2.3% (9/385) of the code review comments are about MISSING CONSTANT USAGE. Constants are often used to indicate the purpose of literals with a specific meaning in a program, e.g., using a constant named `PI` instead of its actual value. Therefore, one of their goals is to improve understandability. In these code reviews, the reviewers pointed out the direct use of literal values (e.g., “It would

be better to declare this Strings as constants” [key=2163]), even when there are constants available (e.g., “...You should also be able to reuse the constants declared at the beginning of ConflictsWithScrutinizer” [key=1687]). By definition, reviewers found these understandability smells in Literals.

Answer to RQ2. In our representative sample of 385 code reviews, we found eight categories of understandability smells. MISSING OR INADEQUATE CODE DOCUMENTATION (86), BAD IDENTIFIER (78), and COMPLEX, LONG, OR INADEQUATE LOGIC (70) comprise more than 60% of the understandability smells in our sample. They are related to 16 source code element types. The three most frequent places, i.e., Method (130), Call (55), and Literal (40), account for more than 58% of the code reviews.

Implications. We discovered a variety of issues affecting code understandability and grouped them into understandability smells. Automated solutions to improve code understandability must deal with these different kinds of issues, with varying levels of context dependence, and tackling multiple modalities: source code as well as unstructured and structured natural language. The results suggest that tools based on LLMs have to be trained in these different languages, as some smells manifest only in Java source code (e.g., UNNECESSARY CODE) whereas others occur in natural language text or identifiers, e.g., BAD IDENTIFIER and INCOMPLETE OR INADEQUATE CODE DOCUMENTATION.

4.3 How likely are understandability improvement comments to be accepted? (RQ3)

To answer this question, we analyzed a set of 385 code review comments related to code understandability improvements (the same set analyzed in Section 4.2) and added another random sample of 385 code review comments not related to code understandability. The code review comments were classified based on two properties: (i) whether they suggested an improvement for understandability, and (ii) whether the suggested changes were accepted or not. The results of this classification are presented in Table 4. This contingency table allows us to investigate the odds of accepting a code review comment, given that it is an understandability improvement.

We verified whether code review comments that include suggestions for improving code understandability are more likely to be accepted by developers than comments that do not include such suggestions. Our analysis revealed that code review comments that suggest understandability improvements are more likely to be accepted than comments that do not address understandability: the odds ratio is 2.69, with a confidence interval [1.71, 4.22] for a 99% confidence level. We also performed a Chi-Square test to evaluate the relationship between these two variables and found a significant correlation between them. In fact, suggestions for improving code understandability were found to be significantly more likely to be accepted than comments unrelated to understandability ($\chi^2(1, N = 440) = 32.9, p < 0.01$).

Additionally, we conducted a detailed analysis of the code reviews that suggested improvements in understandability. We observed that in three instances where understandability improvements were suggested, the developers accepted that the code needed improvement, but

TABLE 4: Frequency of accepted vs. not accepted code review suggestions with and without understandability improvements

Contingency table (770)		Understandability Improvement	
		Presence	Absence
Acceptability	Accepted	323	254
	Not accepted	62	131

they did not agree with the reviewer’s suggestion. Instead, they implemented what they believed was the best alternative without any discussion. For instance, in one code review, the reviewer suggested “*rename module_uuid to assembly_uuid*” [key=1935], but the developer renamed it to `assemblyId`. Also, in eight of the analyzed code reviews developers answered that they accepted the suggestion, but we cannot find any change, even when the developer answered “*changed*” [key=1601]. In 25 instances of suggestions of code understandability improvements that were not accepted, the developers argued in favor of their alternative. For example, during a code review, a reviewer suggests initializing a boolean attribute with the value `false` [key=854], but the developer argues that it is redundant because the default value of a boolean variable is already `false`.

Developers have given a similar level of attention to the understandability smells. For example, there were 71 patches aiming to address BAD IDENTIFIER (Table 5 provides the data for the remaining smells) and 78 instances of that smell (the rightmost column of Table 3). This means that 91% of the instances of this smell resulted in an accepted patch with an improvement, the highest percentage among the understandability smells. Overall, 83.9% (323/385) of the code review comments triggered code understandability improvements. For MISSING CONSTANT USAGE, the percentage was 66.7% (6/9), the lowest among the smells and the most distant from the mean of 82.13%. Calculation of the odds ratio considering the groups where this latter smell was fixed vs. not fixed and where any other smell was fixed vs. not fixed suggests that the difference is not statistically significant (OR=0.37, $p=0.15$).

Answer to RQ3. Code review comments that suggest understandability improvements are significantly more likely to be accepted than comments that do not address understandability improvements. The majority (83.9%) of the suggestions for code understandability improvements in code review comments were accepted. Less than 1% acknowledged the need for improvement but applied another solution. However, in 16.1% of the cases, the pull request authors did not accept the suggestion. In addition, the smells received similar levels of attention, in terms of the percentage of instances for which an understandability improvement was accepted.

Implications. Developers typically accept suggestions for improving code understandability during the code review process. These suggestions and the changes related to them can be valuable and trustworthy data for devising automated approaches to improve code understandability. For example, they can support few-shot learning when leveraging LLMs to make code more readable.

4.4 What code changes are found in understandability improvements? (RQ4)

We examined 323 code review comments from the results of our analysis in Section 4.3 to assess improvements in code understandability. We sorted and categorized the improvements made to address the understandability smells in these reviews, and the results are presented in Table 5. The patches are organized based on the understandability smells presented in Section 4.2, e.g., UNNECESSARY CODE and BAD IDENTIFIER. The similar patches in each category were organized in groups, e.g., $\hookrightarrow$ Unused code. In Table 5, an arrow ($\hookrightarrow$) represents a group and its name is underlined. We also combined the patches based on the action taken by the developer and we created a second level of indentation on the table to avoid repetition of text, e.g., explain the functionality of a method. Patches appearing three or more times are marked with the clubsuit symbol (♣) to indicate their higher frequency. In the remainder of this section, we discuss the understandability improvements that were applied for each understandability smell.

Incomplete or inadequate code documentation. We found 71 patches that added, removed, enhanced, or fixed Javadoc and code comments in the source code. The majority of them (15) are patches that improved the grammar or fixed a typo in existing Javadoc (e.g., “*memory which available*” $\rightarrow$ “*memory which is available*” [key=1115]). Also nine patches added Javadoc documentation to a method or class where it was absent (e.g., methods [key=756] and classes [key=2244]). In other patches related to Javadoc, the developers enhanced the description of code elements (e.g., “*Time spent for waiting a task to be completed.*” $\rightarrow$ “*This time spent while waiting for the task to be completed is being recorded in this counter.*” [key=151]) and added tags and hyperlinks (e.g., “@link JobModel” [key=294]). In terms of changes applied to code comments, there are patches that added code comments to explain the choice of an attribute type [key=2180], removed TODO code comments for completed tasks [key=2387], and transformed a code comment into Javadoc [key=1625]. Finally, four patches added or removed annotation related to documentation, e.g., `Deprecated` [key=2263].

Bad identifier. 71 patches changed an identifier to better communicate what it means in the source code. These patches involved changes regarding both the content and style of the identifiers. To address concerns related to the content of the identifiers, the majority (50) of the patches focused on clarifying their meaning (e.g., `String getDates` $\rightarrow$ `String getDateRange` [key=1042]) and expressing the identifier’s type (e.g., `RequestExecutor requestHandler` $\rightarrow$ `RequestExecutor requestExecutor` [key=340]). Additionally, eleven patches fixed typos (e.g., `verifyEntityTypeMigrationInvalidEntities` $\rightarrow$ `verifyEntityTypeMigrationInvalidEntities` [key=1107]) and four adjusted the identifiers to adhere to the conventions adopted by the project (e.g., `boolean isActive` $\rightarrow$ `boolean active` [key=453]). Furthermore, we observed six patches that aimed to improve the style of the identifiers, such as converting them to lowercase [key=579] or camelcase [key=1753].

Complex, long, or inadequate logic. 54 patches extracted, moved, or replaced complex, lengthy, or inconsistent code in an attempt to simplify it. Among these, five patches extracted parts of a method to reduce its length (e.g., extracting a method). Also, developers applied 16 patches that replaced a usage of an external API or adopted an external API to simplify complex code, such as replacing a chain of method invocations by a unique API method invocation [`key=1700`]. In 14 other scenarios, developers replaced expressions in source code by an alternative that better communicates the functionality of the code, e.g., replacing boolean expression by a call to method `isEmpty()` [key=626]. Additionally, developers applied patches that replace the use of a construct in eight cases, such as replacing an anonymous inner class by a lambda [key=1971] or replacing a lambda with conditional by an `if` statement [key=1892]. Finally, the developers applied five patches to organize the use of APIs in a source code file, such as expanding imports instead of using wildcards [key=2203], and adding an `import` statement to avoid the use of the fully-qualified name of a class (`TileSourceManager.TileSourceTemplate template` $\rightarrow$ `import ...TileSourceManager; private TileSourceTemplate template` [key=1779]).

Unnecessary Code. A total of 43 patches eliminated code that has no impact on the execution. Patches that removed unused code include 24 instances of code elements that are not being referred to (e.g., “*unused constant.*”) and five instances of commented out code [key=444]. Also, the developers applied patches that removed duplicated code (e.g., equivalent methods [key=325]) or processing (e.g., iteration happening twice on the same list [key=536]) 13 times, and different constants with value `null` [key=949]. There are also two patches that inline temporary variables [key=1561].

Inconsistent or disrupted formatting. 37 patches organized and standardized code formatting. These patches manage the use of formatting characters such as space and braces, as well as formatting styles. In 26 of the patches, the developers added space between different code elements (e.g., *horizontal space* [key=493] and *vertical space* [key=2050]); and removed space between related code statements (e.g., *for declaration and the first statement of the block* [key=1660]) or extra space between code elements in the same expression (e.g., *between the point and the invoked method in an expression* [key=1754]). Additionally, developers added other structuring characters (e.g., braces and parentheses) to highlight the region of a code element (e.g., *block* [key=1078] or *operand* [key=1453]). Finally, in eight cases developers changed the order or moved code elements aiming to apply a formatting style, such as keeping the code elements in their own lines (e.g., *body of if statement in its own line* [key=2024]) and arranging method declarations [key=1860].

Wrong, missing, or inadequate string expression or literal. 26 patches addressed concerns related to incorrect, missing, or inadequate string expressions or literals. These patches involve adding, fixing, extending, or changing strings to better suit the code context and improve code understandability. In six of these patches, the developers replaced

TABLE 5: Understandability smell categories and the improvement patches found in the 323 accepted code reviews. The patches with three or more instances are marked with the symbol ♣. In the PDF version, each patch links to an example.

Category ↪ Group or type of improvement patches (# found)
Incomplete or inadequate code documentation (71)
↪ Changes related to Javadoc: (50) - Add new Javadoc block to describe the functionality of a code element (9) ♣ - Add new tag and hyperlink to existing Javadoc (3) ♣ - Add text to existing Javadoc to - explain the functionality of a method (5) ♣ - explain internal implementation details of a method (2) - improve explanation about code element (2) - explain reason for deprecation (1) - Change text in existing Javadoc to - improve grammar or fix a typo (15) ♣ - improve explanation about functionality of a method (6) ♣ - update copyright (3) ♣ - better explain the purpose of an annotation (1) - fix the name of a code element to which it refers (1) - make it more concise (1) - Move a tag of documentation to the conventional place (1) ↪ Changes related to code comments: (17) - Add code comment to - explain an expression (3) ♣ - explain each part of a test (2) - explain the type of an attribute (1) - Remove TODO code comment for tasks that have already been performed (2) - Remove a useless code comment (2) - Change code comment to better explain the associated code element (5) ♣ - Change text in code comment to fix typo (1) - Transform a code comment into Javadoc documentation (1) ↪ Add or remove annotation related to documentation (4) ♣
Bad identifier (71)
↪ Changes related to content: (65) - Modify an identifier to express the meaning or type of an element (50) ♣ - Modify an identifier to fix a typo (11) ♣ - Modify an identifier to be consistent with a convention (4) ♣ ↪ Changes related to style: (6) - Change the style of an identifier to be camelCase, capitalized, or lowercase (6) ♣
Complex, long, or inadequate logic (54)
↪ Extract method (6) ♣ ↪ Extract variable (1) ↪ Rewrite the code to avoid semantic duplicate method call (3) ♣ ↪ Move the code element to consistent place (1) ↪ Changes related to usage of APIs: (16) - Replace method calling chain by existing API (6) ♣ - Use a different type to be consistent with other parts of the code (5) ♣ - Replace usage of an external API by an internal one (2) - Replace a set of statements by direct method in existing API (2) - Replace Lists.newArrayList by Arrays.asList (1) ↪ Changes related to expressions: (14) - Replace an expression by an alternative expression (5) ♣ - Replace call to static method by instance method call (3) ♣ - Replace boolean expression by an alternative direct method (2) - Replace the use of a parameter by an expression with another parameter (1) - Replace indirect expression in parameter by direct value (1) - Replace an expression that creates object by existing attribute (1) - Invert the boolean expression to simplify it (1) ↪ Changes related to constructs: (8) - Replace an imperative logic by functional (3) ♣ - Replace anonymous inner class or method calling chain by lambda (3) ♣ - Replace the while and switch structure by an alternative switch with default (1) - Replace short-circuit in stream by an alternative reducing operation (1) ↪ Changes related to imports: (5) - Expand the imports (3) ♣ - Remove the library name reference in fully qualifier name (2)

Continued on the next column

a natural language string literal by an alternative with a different meaning (e.g., `assertEquals("not_equals"... → assertEquals("Expected and actual values should be the same!")... [key=786]); or a synonym (...are unsupported → ...not supported [key=2287]). Furthermore, in six other patches developers fixed incorrect words in string expressions (Can not`

Continued from the previous column

Category ↪ Group or type of improvement patches (# found)
Unnecessary code (43)
↪ Unused code: (24) - Remove non-referred constant, import, class, method, or variable (19) ♣ - Remove commented out code statements (5) ♣ ↪ Redundant code: (19) - Remove duplicated code or processing (13) ♣ - Remove null constants (1) - Remove type parameter for inferrable type (1) - Remove <code>this</code> (1) - Remove imported type from fully qualified names (1) - Inline temporary (2)
Inconsistent or disrupted formatting (37)
↪ Space usage: (26) - Add or remove horizontal spacing (13) ♣ - Add or remove vertical spacing (13) ♣ ↪ Formatting element visibility: (3) - Add braces in a single statement block (2) - Add parentheses to compound logical expressions (1) ↪ Formatting style: (8) - Add line break to a long statement line (1) - Remove parentheses in logical expression operands and move them to separated lines (1) - Change order of boolean expression operands to make <code>null</code> the second one (1) - Move code elements to their own line (3) ♣ - Move <code>catch</code> declaration to the same line of its associated <code>try</code>'s closing block brace (1) - Move methods to be in call ordering (1)
Wrong, missing, or inadequate string expression or literal (26)
↪ Add a message to an exception (2) ↪ Replace use of the <code>format</code> method by string concatenation (1) ↪ Changes related to a natural language string literal: (18) - Fix an incorrect string literal (6) ♣ - Replace a string literal by a different meaning or a synonymous (6) ♣ - Extend a string literal (3) ♣ - Change literal to adhere to project standards (3) ♣ ↪ Changes related to a string magic value: (5) - Fix an incorrect value (2) - Change magic value to adhere to project standards (3) ♣
Inadequate logging and monitoring (15)
↪ Add logging code (3) ♣ ↪ Remove log (6) ♣ ↪ Change log settings (5) ♣ ↪ Change generic exception to a specific (1)
Missing constant usage (6)
↪ Replace hardcoded value by a new constant (4) ♣ ↪ Use a constant from a library (2)

`set min... → Can't set min...` [key=1149]) and string value (`approveInboxItemByld → approveInboxItemByld` [key=1642]). Additionally, developers changed five direct string values to adhere to project standards [key=2318] or fix values [key=39] to alternatives that adhere to project standards.

Inadequate logging and monitoring. Developers have fixed concerns related to logging and exceptions in 15 patches. In these patches, the developers added new logging instructions in three instances to facilitate the understanding of the execution of a method or statement (e.g., adding logging after processing a list [key=1135]), and removed unnecessary logging in six instances (e.g., removing log for local debugging with `System.out.println` [key=1021]). Moreover, developers changed logging levels (e.g., from *fine* to *finest* [key=1751]) and simplified parameters of formatted text when logging (use direct value instead of variable [key=1173]). Additionally, in one case [key=919], a patch replaced `RuntimeException` by a more specific type, `EncodingException`.

Missing constants. In six patches developers created new constants or used available ones to replace literals. They created new constants to replace literal arguments in four instances (e.g., `bodyParams.put("username",`

user) → bodyParams.put(PARAM_USERNAME, user) [key=2163]) and replaced literals by library constants in two [key=590].

Answer to RQ4. We found 323 patches where developers added, removed, changed, replaced, and moved code elements to improve code understandability. The most common type of understandability improvement was modifying an identifier to express the meaning of a code element (50). This was followed by removing unused constants, imports, classes, methods, or variables (19), changing text in existing Javadoc to improve grammar or fix a typo (15), and removing duplicate code or processing (13).

Implications. These results show that there is considerable variation in understandability improvements: some can be easily supported by linters (e.g., removing unused elements) whereas others require approaches that may be challenging even considering recent advances in machine learning (e.g., removing functionally equivalent code that is not textually similar). They emphasize how diverse tools can be employed to address the different understandability smells.

4.5 To what extent are accepted code understandability improvements reverted? (RQ5)

We analyzed whether or not developers kept the 323 patches analyzed in Section 4.4 throughout the history of the patched files. We observed that none of the patches implementing code understandability improvements were reverted in the pull requests where they were applied. However, we came across seven instances where there are commits with reversions that were undone in subsequent commits of improvement patches in the same pull request. For example, in one case [key=1471] the improvement patch followed a suggestion to remove @Deprecated, but in the following commit this annotation was added again and, later, in subsequent commit, it was removed as suggested initially by reviewer. Also, we discover that 97.2% of the improvement patches were successfully merged. The remaining patches (nine) involved code snippets that were superseded by subsequent commits within the same pull request. For example, a patch added Javadoc to a method and later both the method and the associated Javadoc were removed [key=1338], a method named `inAppDisplayhold` was renamed to `displayPaused` in a patch and later renamed to `autoDisplayPaused` [key=1203], or the patched file was removed entirely [key=416].

We also evaluate whether the (314) patches implementing suggested understandability improvements and integrated into the codebase were reverted at some point in the history of the project after the original pull request and the associated code review were closed. Out of the total 314 patches, 82.8% (260/314) are present in the latest version of the codebase, although in 20 cases, the developers renamed or moved the patched files. However, we cannot find the patch in the last version of the codebase in 54 patches. Among these, the developers replaced the patch with an alternative solution in ten observations. For instance, a subsequent patch changed a snippet with functional implementation (commit `a0c1cbf9746ed9486f08ce4aa2376828e846c786`) by an alternative imperative implementation [key=1453]. In 24 patches, the developers removed the code snippet

TABLE 6: Out of the 323 accepted patches addressing understandability smells, the frequencies of patch merging, reversion within the pull request where it was accepted, permanence in the code based until the last version of the file, and reversion at any point in the history of the project.

Patch status (# analyzed patches)	Yes	No
Patch merged (323)	314 (97.2%)	9 (2.8%)
Patch reverted in the pull request (9)	0 (0%)	9 (100%)
Patch in the last version of the file (314)	260 (82.8%)	54 (17.2%)
Patch reverted in the codebase history (54)	2 (3.7%)	52 (96.3%)

that contained the patch, e.g., the patch included the deprecated annotation and deprecated methods were deleted [key=2263]. Still, in 18 patches, the developers deleted the entire patched file, e.g., `AssemblyJsonServlet.java` deleted [key=1935]. Finally, the patches were reverted and kept reverted until the latest version of the codebase in two of these 54 observations. One such case involved the removal of an unnecessary class [key=2073], but the file was added back into the codebase and later renamed.

As a corollary result of this analysis, we observe that most of the analyzed patches implementing code understandability improvements (97.2%) were integrated into the codebase, whereas Kalliamvakou et al. [37] reported only 44% of pull requests in GitHub (in general) being merged. This stark difference may highlight the high acceptance rate of the reviewers' suggestions for code understandability improvements. It must be taken with a grain of salt, though, as the study of Kalliamvakou et al. was published in 2014. In addition, only two out of 323 patches were reverted and maintained their reversion in the latest codebase version.

Answer to RQ5. After analyzing the history of 323 patches, we discovered that 80.5% (260 out 323) were present in the latest version of the codebase, while the remaining ones were either removed or replaced by alternative solutions. Additionally, we observed that none of the patches were reverted during the pull request. Finally, we found two cases that were removed later during the history of the codebase.

Implications. The results in this section suggest that understandability improvements tend to be stable, i.e., it is unlikely that an improvement will later disappear within a project. Thus, training machine learning models to identify code understandability smells and improving their understandability, one of the goals of code reviews, may yield better results than automating code the whole spectrum of code review activities, including the identification of code with poor understandability and the refactoring of these instances into more readable code. [66], [67].

4.6 Do linters contain rules to detect the identified code understandability smells? (RQ6)

We investigated the coverage of the code understandability smells identified in our study by rules implemented in Spotbugs, PMD, SonarQube, and Checkstyle. To investigate if any of these linters' rules can flag the 323 understandability smells presented in Section 4.4, we manually checked each observation of an understandability smell against each tool to determine if it could detect it. For example, for an observation of UNNECESSARY CODE related to the use of import clauses, we inspected the rules *UnnecessaryImport*,

Unnecessary imports should be removed, and *UnusedImports* in PMD, SonarQube, and Checkstyle respectively, which could detect the *unused import* pointed out by the reviewer. There is no rule to detect this particular instance of UNUSED CODE in Spotbugs. We also considered occurrences where it may be necessary to configure a rule for the linter to detect it. For instance, the rule *MissingJavadocMethod* of Checkstyle needs configuring the scope to warn about missing Javadoc in methods with “package” visibility (otherwise, it only reports issues with “public” ones). Table 7 presents the number of occurrences of each understandability smell that each linter could detect. The rows represent each type of understandability smell, while the columns correspond to each linter. The column *Coverage* presents the number of instances of each smell that can be identified by at least one linter.

Our analysis indicates that 89 out of 323 occurrences of understandability smells can be detected by any of the linters. This highlights wasted human effort, as automated tools could be pointing these issues out. On the one hand, Checkstyle and Sonarqube are responsible for detecting more than half of detectable occurrences. On the other hand, Spotbugs detects only seven occurrences overall, where six are related to UNNECESSARY CODE. Furthermore, we did not find rules that could detect the occurrences related to WRONG, MISSING, OR INADEQUATE STRING EXPRESSION OR LITERAL, either because the tools do not inspect string literals or because they are natural language text.

The understandability smell that can be more often detected in our dataset by the linters is INCONSISTENT OR DISRUPTED FORMATTING. Rules such as *NeedBraces* (Checkstyle), *Lines should not be too long* (SonarQube), and *UselessParentheses* (PMD) can detect the majority (67.6%) of the occurrences of this smell. Additionally, these tools acknowledge that different projects have different styles. For example, in a code review comment, the reviewer suggested that “*bracket [left curly brace] should be on the next line*” [key=1487], i.e., the left curly brace should be in beginning a new line instead of the end of current statement line. Accounting for the possibility of some projects using left curly braces at the end of the current line, SonarQube contains rules for both cases, *An open curly brace should be located at the beginning of a line* and *An open curly brace should be located at the end of a line*.

The two most popular understandability smells, BAD IDENTIFIER and INCOMPLETE OR INADEQUATE CODE DOCUMENTATION, are the second and fourth less detectable by linters, respectively. The majority of the occurrences of these understandability smells are associated with natural language text (e.g., typo in identifier and issue in content of Javadoc). The cases of BAD IDENTIFIER detected by the linters are those related to style (e.g., parameters starting with capital letter [key=579]). In these cases, rules such as *FormalParameterNamingConventions* (PMD), *Nm: Field names should start with a lower case letter* (NM_FIELD_NAMING_CONVENTION) (Spotbugs), and *LocalVariableName* (Checkstyle) can be used to detect them. In the case of INCOMPLETE OR INADEQUATE CODE DOCUMENTATION, there are only rules for missing documentation, e.g., *CommentRequired* (PMD); missing or incorrect order of tags in Javadoc, e.g., *WriteTag* (Checkstyle); and

TODO code comments, e.g., *Track uses of “TODO” tags* (SonarQube). Many instances of these smells require an in-depth understanding of the associated elements, e.g., to produce an identifier that better represents the purpose of a method or to improve an explanation about an element. At the same time, the most common change applied to Javadoc documentation in our study is fixing typos (Table 5), which could be automated.

Among the remaining 234 out of 323 instances of understandability smells, we identify occurrences that could be covered by the investigated linters by performing small adaptations to existing rules. For example, the linters could cover the *missing parentheses* (e.g., key=1453) by creating an inverse rule to the *UselessParentheses* rule of PMD or “*UnnecessaryParentheses*” rule of Checkstyle. This new rule could suggest the inclusion of additional parentheses in boolean expressions to highlight evaluation order. Additionally, the linters could identify opportunities to use primitive types instead of the corresponding boxed types (e.g., key=1201). As another example, the linters could detect when an exception is created without a message (e.g., key=1345). However, other instances of understandability smells would require more sophisticated analyses. For example, to detect the occurrences of incomplete documentation, unnecessary logging messages, confusing string messages, inadequate use of method call chains, and missing logging, it would be necessary to understand the context of the source code, the culture of the project, the intention of the developers, predict the system behavior at run time, or to work with both Java and natural languages.

Answer to RQ6. We found 89 out of 323 occurrences of understandability smells that could be detected by the linters. Some of the remaining 234 occurrences of understandability smells could be detected by creating new rules in these linters, while other occurrences require a deeper and context-dependent analysis that goes beyond what linters typically do.

Implications. The four linters combined have rules to cover less than 30% of the instances of understandability smells from our dataset. This suggests that developers waste effort during code reviews by alluding to issues that existing automated tools could detect. Among the cases that the linters cannot detect, many of them require an in-depth “understanding” of the program under analysis or the context in which it is inserted. In particular, current linters are unable to cope with code understandability issues related to natural language. Some of these cases present opportunities to devise new tools, e.g., to better summarize the intent of a code element or to give it a more illustrative name. Others could be addressed by simple improvements to current linters, e.g., pointing out typos in documentation, identifiers, and string literals. This also applies to rules related to other aspects, such as messages in exceptions and the use (or not) of boxed types.

5 THREATS TO VALIDITY

In this section, we discuss the threats to the validity of our study.

TABLE 7: The frequency of understandability smells that are covered by linters.

Understandability smells (# occurrences)	Spotbugs	PMD	SonarQube	Checkstyle	Coverage
INCOMPLETE OR INADEQUATE CODE DOCUMENTATION (71)	0 (0.0%)	10 (14.1%)	7 (9.9%)	14 (19.7%)	14 (19.7%)
BAD IDENTIFIER (71)	1 (1.4%)	6 (8.5%)	3 (4.2%)	5 (7.0%)	7 (9.9%)
COMPLEX, LONG OR INADEQUATE LOGIC (54)	0 (0.0%)	8 (14.8%)	11 (20.4%)	7 (13.0%)	16 (29.6%)
UNNECESSARY CODE (43)	6 (14.0%)	11 (25.6%)	18 (41.9%)	7 (16.3%)	22 (51.2%)
INCONSISTENT OR DISRUPTED FORMATTING (37)	0 (0.0%)	3 (8.1%)	14 (37.8%)	25 (67.6%)	25 (67.6%)
WRONG, MISSING, OR INADEQUATE STRING EXPRESSION OR LITERAL (26)	0 (0.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)
INADEQUATE LOGGING AND MONITORING (15)	0 (0.0%)	2 (13.3%)	2 (13.3%)	0 (0.0%)	2 (13.3%)
MISSING CONSTANT USAGE (6)	0 (0.0%)	0 (0.0%)	3 (50.0%)	3 (50.0%)	3 (50.0%)
Total (323)	7 (2.2%)	40 (12.4%)	58 (18.0%)	61 (18.9%)	89 (27.6%)

Selection of projects. We constructed our dataset by mining software repositories from GitHub. However, researchers must be cautious about potential pitfalls [37]. To ensure data quality, we applied filters to exclude personal projects, repositories with small contributor groups, and forked repositories. Moreover, we utilized a heuristic to select projects with an active code review process during a specific period. Also, we manually analyzed each remaining repository to guarantee the relevance of the dataset, excluding non-software repositories. Additionally, some projects may have configured an ASAT (Automated Static Analysis Tool) to prevent certain issues related to code understandability from being introduced into the codebase. However, since we did not assess whether any project adopted this approach, it is possible that some understandability smells may not have been identified by our study.

Size of the sample of code review comments. Before analyzing our sample of 2,401 code review comments to determine whether they are about code understandability improvements, we explored various natural language processing techniques, such as LDA, GSDMM, LSA, and LDA Guided, to extract topics and create clusters for both understandability and non-understandability improvements. Unfortunately, we encountered challenges in obtaining satisfactory results due to two main reasons: (i) our training sample was relatively small, both in terms of the number of comment instances and the length of their text, and (ii) there were significant similarities between comments related and unrelated to code understandability. This problem has been reported by previous work as well [68], [69].

Manual analysis and subjectivity. We conducted a study where we manually evaluated 2,401 code review comments. Manual categorization may result in bias due to different interpretations by human coders. Some comments that were analyzed lack precision and clarity. This could affect the validity of any analysis or decision-making based on the text. Additionally, it is possible that there is overlap between different smells. In these cases, we analyze the reviewers' comments to understand the intent of the proposed change. Each comment was independently classified by at least two authors, and whenever a disagreement was encountered, up to five discussed the classification. We also held sessions to discuss dubious comments and to align the classification process. Furthermore, we analyzed the source code to which each comment refers and subsequent changes to it that may have been triggered by the comment.

We also conducted a manual analysis of rules in four linters that could potentially detect understandability smells highlighted by reviewers. For each instance of an understandability smell that we analyzed, we searched through a

set of 1,315 rules to find the ones that could identify the smell. Some of these rules can be configured to detect a smell, and we considered all possible configurations. However, we decided not to include rules that relied on regex patterns due to their specificity. It is important to note that there may be some rules that we missed during this process, which could result in some issues being classified as not covered by linters.

Code review bots. During the code review process for pull requests, we came across bots that inspect the source code, i.e., bots that work as linters. These bots, like `vaadin-bot` and `codeclimate`, identify potential issues based on pre-configured rules. For instance, `vaadin-bot`¹⁰ found a deprecated method annotation and suggested removing it in the future, while `codeclimate`¹¹ commented on a method that exceeded the allowed number of lines and suggested refactoring. `Codacy`, `Lint-staged`, and `Prettier` are examples of popular bots that review the source code searching for violations of their pre-configured rules. We decided not to investigate these bots specifically, considering that their rules are similar to the linters studied previously. For example, `codeclimate` leverages `SonarJava`¹², also used by `SonarQube`, to verify Java programs.

External Validity. This study focused exclusively on open-source projects, and as such, the findings may not be fully representative of all software development contexts. To address this concern, we made efforts to include a diverse range of software projects in our analysis. Nonetheless, it is essential to acknowledge that the identified code understandability smells and corresponding understandability improvements might not be universally applicable across all programming languages, frameworks, development environments, or development teams. Furthermore, a limitation inherent to using GitHub is that many engineered projects in the platform are mostly developer-centric, e.g., libraries, frameworks, etc. This means that many sub-domains, such as games and mobile apps, are under-represented. Additionally, our study focused on code reviews conducted in 2020, which may have been impacted by COVID-related issues. Therefore, it is unclear how much we can generalize the findings. In our analysis, we did not take into account the experience level of the developers and reviewers. This means that we may have factored in the opinions of inexperienced individuals. However, we carefully selected a diverse sample group that included many developers and

10. https://github.com/vaadin/flow/pull/8409#discussion_r429865662

11. https://github.com/triplea-game/triplea/pull/7241#discussion_r460364607

12. <https://docs.codeclimate.com/docs/sonar-java>

reviewers. This helped to minimize the impact of this issue.

6 RELATED WORK

In this section, we examine related work. We start by contrasting the understandability smells and improvements we have identified in practice with recommendations and experimental results reported in the literature (Section 6.1). This helps us identify mismatches and accordance between research and practice, as well as research gaps. We then review previous work discussing models to assess code understandability (Section 6.2), following up with papers that have investigated manifestations of confusion and reactions to lack of necessary information during code reviews (Section 6.3).

6.1 Coding alternatives for understandability: literature vs. practice

We found in the literature many studies about code understandability. Among these studies, the researchers investigated the best code alternatives for code understandability. Oliveira et al. [70] conducted a systematic literature review that categorized and organized studies that used empirical evaluation to compare code alternatives for formatting elements. For instance, Gopstein et al. [18], Medeiros et al. [7], and Langhout and Aniche [9] examined the impact of explicit block delimiters (i.e., curly braces in Java) compared to the omission of block delimiters when possible. Furthermore, we found studies that compared other code elements, such as monolithic expressions compared to multiple shorter expressions [5]. We aim to compare our findings from practical experience with what we gathered from these studies.

Our focus in this paper is on understandability improvements, i.e., situations where existing code exhibits an understandability smell and is replaced by an alternative solution that is more understandable. Thus, in this section, we focus on the five understandability smells where, to the best of our knowledge, alternative solutions were compared in existing studies from a human-centric perspective, e.g., based on the preference or performance of study participants: BAD IDENTIFIER, UNNECESSARY CODE, INCONSISTENT OR DISRUPTED FORMATTING, COMPLEX, LONG, OR INADEQUATE LOGIC, and MISSING CONSTANT USAGE.

BAD IDENTIFIER. Chaudhary and Sahasrabuddhe [71] evaluated the impact on code understandability of using identifier names unrelated to the program domain. This study concluded that the identifier names with meaning related to the problem domain facilitate understanding. In this direction, the practitioners improve the identifier names to better match their intent and type. Santos and Gerosa [5] compared using fully-qualified names and `import` clauses with shorter type names. They found out that participants preferred to avoid fully-qualified names and add `import` clauses, which is what the practitioners in our study prefer. Schankin et al. [72] investigated whether a more descriptive name (e.g., *singleWordParts*) is easier to read than a single-word one (e.g., *parts*). They found that a descriptive name was the best in one scenario, and there was no difference

in the other. The practitioners also considered improving understandability with descriptive names. Sharif and Maletic [73] and Binkley et al. [74] investigated the effect of the style of the identifier (underscore and camel case) on code understandability. Sharif and Maletic found that the use of an underscore to separate parts of an identifier improves understandability, whereas Binkley et al. found out that camel case works better. For practitioners in Java, using the camel case style for non-constant identifiers improves code understandability.

UNNECESSARY CODE. Gopstein et al. [18] and Torres et al. [35] investigated whether the presence of dead, unreachable, or repeated code (e.g., `v1 = 1; v1 = 2`), leads to misunderstanding in C and JavaScript programs, respectively. Both studies found that the presence or absence of this pattern did not significantly impact code understandability. In contrast, practitioners seem to believe this is an important issue in practice: 13.3% of all the 323 analyzed code review comments pertain to UNNECESSARY CODE. The practitioners removed the unnecessary code, commented out source code, and duplicated code to improve code understandability.

INCONSISTENT OR DISRUPTED FORMATTING. Santos and Gerosa [5], and Sampaio and Barbosa [75] evaluated whether different conventions in terms of vertical and horizontal space between the code elements impact code understandability and did not find significant differences. Conversely, practitioners consider that spacing is an issue worth raising during code review. Out of the 323 analyzed comments, 26 (8%) pertain to spacing. Sometimes, they remove space, and others add space. Sykes et al. [76], Sampaio and Barbosa [75], Gopstein et al. [18], Langhout and Aniche [9], Torres et al. [35], and Medeiros et al. [7] investigated the presence or omission of the block delimiter (e.g., curly braces in Java) in blocks with one statement. Gopstein et al., Torres et al., Langhout and Aniche, and Medeiros et al. found that the presence of the block delimiter improves code understandability (in C, JavaScript, and Java). The other researchers did not find any difference. The practitioners preferred to use block delimiters even in a one-statement block. Geffen and Maoz [77] investigated the effect of ordering methods in understanding the code. For some cases ordering considering the order of calling and the connectivity between the methods is the best for code understandability. The practitioners consider that the calling order improves code understandability.

COMPLEX, LONG, OR INADEQUATE LOGIC. Gopstein et al. [18], Medeiros et al. [7], Torres et al. [35], Costa et al. [78], and Langhout and Aniche [9] investigated which one promotes better understandability: the conditional operator or `if` statements, in C, JavaScript, Python, and Java small programs. Gopstein et al. and Medeiros et al. found that `if` statements improve code understandability (in C programs) whereas Langhout and Aniche, Costa et al., and Torres et al. did not find a statistically significant difference. The practitioners considered both `if` statement and ternary operator to improve code understandability in distinct scenarios. Lucas et al. [79] compared the use of lambdas and anonymous inner classes based on the opinions of 28 ex-

perienced Java developers. Their results are aligned with our findings when examining code review comments: practitioners prefer lambda expressions. Most of the instances of the COMPLEX, LONG, OR INADEQUATE understandability smell are either too specific to be studied in a more general context, e.g., object creation that can be avoided by using an existing attribute, or, to the best of our knowledge, have not been studied, e.g., using `try` statements that manage resources, instead of `try` with resources.

MISSING CONSTANT USAGE. Gopstein et al. [18], Langhout and Aniche [9], and Torres et al. [35] evaluated the effect of using constant variables instead of direct values on code understandability. They did not find a statistically significant difference. The practitioners created and used constants instead of direct values to improve code understandability.

6.2 Estimating and classifying code understandability

Piantadosi et al. [26] investigated the frequency, methods, and reasons for changes in code understandability (*code readability* in their paper) during the evolution of open-source software. They built a model to define states (non-existing, other-name, readable, and unreadable) in which a file can be in at a given time. Using this model, they analyzed the transitions related to code understandability in the commit history of 25 open-source software projects. By analyzing more than 340,000 understandability transitions, they discovered that code understandability rarely changes during the evolution of the project. In a manual analysis of 57 understandability transitions, they found that improving or decreasing code understandability is mostly unintentional.

Fakhoury et al. [27] investigated whether state-of-the-art understandability models are able to capture understandability (*readability* in their paper) improvements as explicitly tagged by open source developers in commit messages. The authors analyzed 548 commits from 63 engineered Java projects that explicitly mention understandability improvements in the commit message. They then used three state-of-the-art understandability models to measure how much these commits affected code understandability. They found that the models did not capture understandability improvements in most cases. Additionally, they used static analysis tools to investigate what type of understandability changes were applied in the commits related and unrelated to code understandability. They discovered that the former frequently fix concerns related to imports, white spaces, and braces, whereas the latter tends to introduce them.

Roy et al. [28] proposed a model to detect code understandability (*readability* in their paper) improvements made by developers in real-world scenarios using commit histories from open-source software projects by extending the dataset of Fakhoury et al. [27]. They also included commits unrelated to code understandability improvements to create an oracle. The authors employed seven source code analysis tools to collect metrics before and after each commit at the file level. By comparing these metrics, they identified key differences that served as features for their model. The model achieved a precision of 79.2% and a recall of 67% on the test set.

Buse and Weimer [21] proposed a method for measuring code understandability (*code readability* in their paper) based on human notions of understandability. They collected feedback from 120 human annotators on their understanding of 100 code snippets, correlating this feedback with source code features. This data was then used to train a machine learning algorithm to classify code understandability. The resulting model predicted understandability judgments with an 80% success rate. Posnett et al. [80] proposed a simplification of Buse and Weimer’s [21] model. They leverage source code size metrics and Halstead metrics [81]. Using the dataset of Buse and Weimer [21] to train and evaluate their model, they obtained better results using just three features in small code snippets: volume, entropy, and number of lines.

Scalabrino et al. [15] analyzed the correlation between 121 metrics related to code, documentation, and developers and six different proxies of code understandability, based on 444 human evaluations. The study revealed that none of the individual metrics strongly correlated with the proxies for code understandability. However, combining multiple state-of-the-art metrics resulted in models that showed a slight improvement in the regression performance of two proxies, Timed Actual Understandability (Correlation: +0.07; Mean absolute error: -0.02) and Actual Understandability (Correlation: +0.02; Mean absolute error: +0.00).

Lavazza et al. [82] evaluated whether code measures are correlated with code understandability. They conducted an empirical study with students who performed maintenance tasks. The results showed that a code comprehensibility model cannot rely solely on measures of code structure. The obtained models were not very accurate, with the average prediction error being around 30%.

These studies [15], [21], [80], [82] have introduced models to quantitatively estimate the degree of code understandability or to detect code understandability improvements [26], [28]. In contrast, our work aims to delve into the code constructs and practices that enhance code understandability, specifically exploring code alternatives suggested by code quality specialists during software development. Additionally, we manually identified code understandability changes using code review comments rather than relying on an automatic approach based on metrics or keywords [26], [27], [28]. Furthermore, we categorized code understandability smells based on the associated issues and solutions instead of categorizing them in terms of software maintenance types [26] or rules of static analysis tools [27]. Finally, our work acknowledges that different projects have different rules, priorities, and development cultures and these differences have an impact on what constitutes readable code.

Some other studies [83], [84], [85] have proposed techniques for automating assessment through advanced methods, including convolutional neural networks. Mi et al. [83] introduced IncepCRM, a model that autonomously extracts multi-scale features from code, aided by human annotations to enhance accuracy. In a distinct approach [84], an 83.8% accuracy is achieved, surpassing prior machine-learning models. This is realized by transforming source code into matrices for convolutional neural networks, culminating in DeepCRM, an architecture of three networks trained on

diverse preprocessed data. Similarly, in another study [85], a graph-based method is adopted, parsing source code into a graph containing abstract syntax tree (AST), along with control and data flow edges, preserving semantic structural insights. This approach attains 72.5% and 88% accuracy in three-class and two-class classifications, respectively. Our analysis of inline code reviews, focusing on developers' perspectives regarding code understandability, provides valuable insights for the referenced works. The research conducted by Mi et al. [83] could further benefit by incorporating insights from our study to refine their IncepCRM model. Our findings might aid in identifying specific aspects of code understandability highlighted during reviews, enabling more effective feature extraction for multi-scale representation. Similarly, the strategy proposed by the author in [84] could gain from our insights by considering the nuances of developer concerns about code understandability in the design of novel representation strategies. Integrating our unique dataset and analytical framework could enhance these approaches, contributing to advancements in code understandability evaluation.

Vitale et al. [86] introduced an approach to improve code understandability (*code readability* in their paper). They analyzed the revision history to identify commits aimed at improving understandability and collected a dataset of 122k such commits. They used the T5 model to emulate developers' actions in enhancing code understandability. Their findings showed that their approach accurately identified understandability-improving commits around 86% of the time. However, the model's precision in suggesting actions to improve code understandability ranged between 21% and 28%. Our findings lay the groundwork for the development of such predictive models. For example, we identified understandability smells that are currently not tackled by existing linters and therefore require more powerful approaches such as the use of LLMs. Furthermore, the dataset of understandability smells and improvements we have produced can be leveraged to fine-tune future language models.

More generic LLM models could leverage our insights to enhance their predictive capabilities. These models could refine their feature extraction processes by considering the specific factors that developers prioritize for code understandability. Incorporating our findings into their frameworks could result in more accurate assessments of code understandability, aligning more closely with real-world developer concerns. Thus, our work offers a valuable resource for improving the effectiveness of broader LLM-based approaches to code evaluation. CodeBERT, introduced in [87] by Zhangyin Feng et al., is designed for tasks like code summarization and code-to-text generation. Analyzing inline reviews for understandability can offer valuable insights into the patterns and priorities developers have regarding code understandability. By integrating these insights, CodeBERT's performance can be enhanced, especially in generating human-readable code summaries and comments. Understanding the understandability aspects developers value most can enable models like CodeBERT to align more closely with human preferences in code generation and summarization.

6.3 Understandability and Code Reviews

In this section, we present studies that examine code review comments and how developers express confusion in these comments while reading code. The work by Dantas et al. [29] is arguably the one that is the most related to our study. They evaluated code understandability (*code readability* in their paper) improvements in the context of pull requests (PRs) and compared them with the rules of SonarQube. Dantas and colleagues utilized keywords such as "readability" and "understandability" and analyzed the context of proposed code changes to identify PRs that enhance code understandability. The researchers identified and categorized 284 PRs and found 26 different types of code understandability improvements. Among the 284 code understandability improvements classified, they found that 26 of them are detected by SonarQube. Differently, our study examined 2,401 code review comments that suggested fine-grained source code improvements. We manually identified 1,012 code review comments that suggested code understandability improvements based on the reviewers' intent. We classified the understandability smells and their solutions, as well as investigated the prevalence of the patches applied to improve code understandability. We have also analyzed whether understandability improvements are accepted or not and whether they are reversed. Finally, we analyzed the potential of four different linters to detect the understandability smells.

The work of Pascarella et al. [88] aimed to understand the information necessary for reviewers to conduct a proper code review. It suggests ways in which research and tool support can enhance the effectiveness and efficiency of developers in their role as reviewers. The study analyzed 900 code review comments from three large open-source projects and identified seven primary information needs of reviewers, such as understanding the uses of methods and variables declared or modified within the code. In a similar vein, Ebert et al. [60] investigated the causes and consequences of confusion in code reviews and how developers typically handle such issues. The study involved a survey of 54 valid responses and a manual evaluation of 307 code review comments (both inline and general). The authors identified 30 sources of confusion, 14 ways that confusion impacts code review, and 13 coping strategies to deal with confusion in code reviews. The research revealed that developers often experience confusion during code reviews when dealing with long or complex code changes and changes that address multiple issues.

Our work differs from previous studies because we aim to evaluate how developers improve understandability through code review comments. Furthermore, the findings of our study can be used as recommendations to developers to improve their source code. This can lead to code that causes less confusion when being reviewed.

Other studies have explored approaches to automate code review, in the generation of patches [89], [90] and of code review comments [91]. The former approach leverages LLMs and the latter employs information retrieval. Neither of them attempts to distinguish the purposes of the code review comments. Likewise, the study by Fregnan et al. [92] evaluated what extent a machine learning-based

technique can automatically classify review changes. The researchers classified 1,504 review changes manually and then evaluated three different machine learning algorithms to see if they could classify review changes automatically at two different levels of detail. Their findings indicate that machine learning can be effectively used to classify review-induced changes. This paper is complementary to these approaches in that it narrows down the scope of investigation to code understandability improvements. It shows the relevance of code review comments aiming to improve understandability, highlights the high-level problems that these comments address, shows how many of the corresponding improvements materialize, and indicates that they are often accepted and rarely reversed. It also provides evidence that automating the application of improvements involves a variety of issues, targets different languages, natural and artificial, and in some cases can be solved easily whereas in others requires a deeper understanding.

7 CONCLUSION

Code understandability is crucial for efficient software development and maintenance, as it reduces the time and cost required for these processes. Despite its importance, improving code understandability is not straightforward, as the factors influencing it are not fully understood. Previous studies comparing different coding approaches have yielded inconclusive or controversial results. Additionally, style guides often do not align with experimental findings, leading to differing perceptions among developers.

To address this research gap, this study focused on code review comments in open-source projects, where developers actively strive to improve code understandability. The analysis of 2,401 code review comments identified 1,012 comments related to code understandability improvements. This highlights the significance of code review in enhancing code understandability. A deeper analysis of 385 code review comments identified eight categories of code understandability smells, such as INCOMPLETE OR INADEQUATE CODE DOCUMENTATION and BAD IDENTIFIERS. Developers often accepted the suggested improvements and rarely reverted them. The study also explored the types of patches applied to address code understandability smells, where 97.2% of applied patches were integrated into the project's codebase. Additionally, while some of these issues can be readily detected and fixed by linters, others demand more intricate analysis, considering natural language components and project context.

This study contributes to the understanding of developers' concerns and actions regarding code understandability during code reviews. The dataset of code review comments serves as a valuable resource for researchers and developers, enabling the creation of automated tools to detect and repair code understandability smells. Such tools would allow developers to focus on other critical aspects of code review, promoting correctness and identifying security vulnerabilities.

ACKNOWLEDGEMENTS

This work was partially supported by INES (www.ines.org.br), CNPq grant 465614/2014-0, FACEPE

grants APQ-0399-1.03/17 and APQ/0388-1.03/14, CAPES grant 88887.136410/2017-00, the Swedish Foundation for Strategic Research (SSF), and the Wallenberg Artificial Intelligence, Autonomous Systems and Software Program (WASP) funded by Knut and Alice Wallenberg Foundation.

REFERENCES

- [1] J. Siegmund, "Program comprehension: Past, present, and future," in *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, vol. 5, 2016, pp. 13–20.
- [2] R. Minelli, A. Mocci, and M. Lanza, "I know what you did last summer - an investigation of how developers spend their time," in *23rd ICPC*, 2015.
- [3] X. Xia, L. Bao, D. Lo, Z. Xing, A. E. Hassan, and S. Li, "Measuring program comprehension: A large-scale field study with professionals," *IEEE Transactions on Software Engineering*, vol. 44, no. 10, pp. 951–976, 2017.
- [4] D. Oliveira, R. Bruno, F. Madeiral, and F. Castor, "Evaluating code readability and legibility: An examination of human-centric studies," in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2020, pp. 348–359.
- [5] R. M. a. dos Santos and M. A. Gerosa, "Impacts of Coding Practices on Readability," in *ICPC 2018*, 2018.
- [6] S. Fakhoury, D. Roy, Y. Ma, V. Arnaoudova, and O. Adesope, "Measuring the impact of lexical and structural inconsistencies on developers' cognitive load during bug localization," *EmSE*, vol. p. August 2019.
- [7] F. Medeiros, G. Lima, G. Amaral, S. Apel, C. Kästner, M. Ribeiro, and R. Gheyi, "An investigation of misunderstanding code patterns in C open-source software projects," *EmSE*, vol. 24, no. 4, p. August 2019.
- [8] E. S. Wiese, A. N. Rafferty, and A. Fox, "Linking Code Readability, Structure, and Comprehension among Novices: It's Complicated," in *ICSE-SEET 2019*. 2019, p.
- [9] C. Langhout and M. Aniche, "Atoms of confusion in java," in *2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC)*. IEEE, 2021, pp. 25–35.
- [10] G. Scanniello and M. Risi, "Dealing with Faults in Source Code: Abbreviated vs. Full-Word Identifier Names," in *ICSM 2013*. 2013, p.
- [11] A. Bacchelli and C. Bird, "Expectations, outcomes, and challenges of modern code review," in *2013 35th International Conference on Software Engineering (ICSE)*, 2013, pp. 712–721.
- [12] C. Sadowski, E. Söderberg, L. Church, M. Sipko, and A. Bacchelli, "Modern code review: A case study at google," in *International Conference on Software Engineering, Software Engineering in Practice track (ICSE SEIP)*, 2018.
- [13] A. Cunha, T. Conte, and B. Gadelha, "Code review is just reviewing code? a qualitative study with practitioners in industry," in *Proceedings of the XXXV Brazilian Symposium on Software Engineering*, ser. SBES '21. New York, NY, USA: ACM, 2021, p. 269–274. [Online]. Available: <https://doi.org/10.1145/3474624.3477063>
- [14] "Pull request #10182 - azure/azure-sdk-for-java," https://github.com/Azure/azure-sdk-for-java/pull/10182/#discussion_r409002709, accessed: 2023-08-05.
- [15] S. Scalabrino, G. Bavota, C. Vendome, M. Linares-Vasquez, D. Poshvanyk, and R. Oliveto, "Automatically assessing code understandability," *IEEE Transactions on Software Engineering*, vol. 47, no. 3, pp. 595–613, 2019.
- [16] S. N. Woodfield, H. E. Dunsmore, and V. Y. Shen, "The Effect of Modularization and Comments on Program Comprehension," in *ICSE 1981*. 1981, p.
- [17] T. Tenny, "Program Readability: Procedures Versus Comments," *TSE*, vol. 14, no. 9, p. September 1988.
- [18] D. Gopstein, J. Iannaccone, Y. Yan, L. DeLong, Y. Zhuang, M. K.-C. Yeh, and J. Cappos, "Understanding Misunderstandings in Source Code," in *ESEC/FSE 2017*. 2017, p.
- [19] J. C. Hofmeister, J. Siegmund, and D. V. Holt, "Shorter Identifier Names Take Longer to Comprehend," *EmSE*, vol. 24, no. 1, p. February 2019.

- [20] S. Scalabrino, G. Bavota, C. Vendome, M. Linares-Vásquez, D. Poshyvanyk, and R. Oliveto, "Automatically assessing code understandability: How far are we?" in *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2017, pp. 417–427.
- [21] R. P. Buse and W. R. Weimer, "Learning a metric for code readability," *IEEE Transactions on Software Engineering*, vol. 36, no. 4, pp. 546–558, 2010.
- [22] "Pull request #571 - twilio/twilio-java," https://github.com/twilio/twilio-java/pull/571#discussion_r489083957, accessed: 2023-08-05.
- [23] M. Smit, B. Gergel, H. J. Hoover, and E. Stroulia, "Code Convention Adherence in Evolving Software," in *Proceedings of the 27th IEEE International Conference on Software Maintenance (ICSM '11)*. USA: IEEE Computer Society, 2011, p. 504–507.
- [24] J. Cohen, "A Coefficient of Agreement for Nominal Scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960. [Online]. Available: <https://doi.org/10.1177/001316446002000104>
- [25] B. A. Kitchenham, D. Budgen, and P. Brereton, *Evidence-Based Software Engineering and Systematic Reviews*. Chapman & Hall/CRC, 2015.
- [26] V. Piantadosi, F. Fierro, S. Scalabrino, A. Serebrenik, and R. Oliveto, "How does code readability change during software evolution?" *Empirical Software Engineering*, vol. 25, pp. 5374–5412, 2020.
- [27] S. Fakhoury, D. Roy, A. Hassan, and V. Arnaoudova, "Improving source code readability: Theory and practice," in *2019 IEEE/ACM 27th International Conference on Program Comprehension (ICPC)*, 2019, pp. 2–12.
- [28] D. Roy, S. Fakhoury, J. Lee, and V. Arnaoudova, "A model to detect readability improvements in incremental changes," in *28th ICPC*, 2020, p. 25–36.
- [29] C. Dantas, A. Rocha, and M. Maia, "How do developers improve code readability? an empirical study of pull requests," in *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2023.
- [30] C. Y. Chong, P. Thongtanunam, and C. Tantithamthavorn, "Assessing the students' understanding and their mistakes in code review checklists: An experience report of 1,791 code review checklist questions from 394 students," in *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*, 2021, pp. 20–29.
- [31] A. Gawande, *The Checklist Manifesto: How to Get Things Right*. Profile books, 2011.
- [32] G. D. Israel *et al.*, "Determining sample size," 1992.
- [33] R. Pawlak, M. Monperrus, N. Petitprez, C. Noguera, and L. Seinturier, "Spoon: A Library for Implementing Analyses and Transformations of Java Source Code," *Software: Practice and Experience*, vol. 46, pp. 1155–1179, 2015. [Online]. Available: <https://hal.archives-ouvertes.fr/hal-01078532/document>
- [34] C. Brown and C. Parnin, "Understanding the impact of github suggested changes on recommendations between developers," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2020, pp. 1065–1076.
- [35] A. Torres, C. Oliveira, M. V. Okimoto, D. Marcilio, P. Queiroga, F. Castor, R. Bonifácio, E. D. Canedo, M. Ribeiro, and E. Monteiro, "An investigation of confusing code patterns in javascript," *J. Syst. Softw.*, vol. 203, p. 111731, 2023.
- [36] D. D. Nucci, F. Palomba, D. A. Tamburri, A. Serebrenik, and A. D. Lucia, "Detecting code smells using machine learning techniques: Are we there yet?" in *25th International Conference on Software Analysis, Evolution and Reengineering, SANER 2018, Campobasso, Italy, March 20-23, 2018*. IEEE Computer Society, 2018, pp. 612–621.
- [37] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, "The promises and perils of mining github," in *Proceedings of the 11th working conference on mining software repositories*, 2014, pp. 92–101.
- [38] J. Shimagaki, Y. Kamei, S. McIntosh, D. Pursehouse, and N. Ubayashi, "Why are commits being reverted?: a comparative study of industrial and open source projects," in *2016 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2016, pp. 301–311.
- [39] C. Sadowski, J. van Gogh, C. Jaspan, E. Söderberg, and C. Winter, "Tricorder: Building a program analysis ecosystem," in *37th IEEE/ACM International Conference on Software Engineering, ICSE 2015, Florence, Italy, May 16-24, 2015, Volume 1*. IEEE Computer Society, 2015, pp. 598–608.
- [40] C. Vassallo, S. Panichella, F. Palomba, S. Proksch, H. C. Gall, and A. Zaidman, "How developers engage with static analysis tools in different contexts," *Empirical Software Engineering*, vol. 25, pp. 1419–1457, 2020.
- [41] V. Lenarduzzi, F. Pecorelli, N. Saarimaki, S. Lujan, and F. Palomba, "A critical comparison on six static analysis tools: Detection, agreement, and precision," *Journal of Systems and Software*, vol. 198, p. 111575, 2023.
- [42] "Spotbugs - bug descriptions," <https://spotbugs.readthedocs.io/en/stable/bugDescriptions.html>, accessed: 2023-08-05.
- [43] "Pmd - java rules," https://docs.pmd-code.org/latest/pmd_rules_java.html, accessed: 2023-08-05.
- [44] "Sonarqube - java static code analysis," <https://rules.sonarsource.com/java/>, accessed: 2023-08-05.
- [45] "Checkstyle - checks," <https://checkstyle.sourceforge.io/checks.html>, accessed: 2023-08-05.
- [46] D. Stefanović, D. Nikolić, D. Dakić, I. Spasojević, and S. Ristić, "Static code analysis tools: A systematic literature review." *Annals of DAAAM & Proceedings*, vol. 7, no. 1, 2020.
- [47] K. F. Tómasdóttir, M. F. Aniche, and A. van Deursen, "The adoption of javascript linters in practice: A case study on eslint," *IEEE Trans. Software Eng.*, vol. 46, no. 8, pp. 863–891, 2020.
- [48] O. Dabic, E. Aghajani, and G. Bavota, "Sampling projects in github for MSR studies," in *18th IEEE/ACM International Conference on Mining Software Repositories, MSR 2021*. IEEE, 2021, pp. 560–564.
- [49] R. M. Groves, F. J. F. Jr., M. P. Couper, J. M. Lepkowski, E. Singer, and R. Tourangeau, *Survey Methodology*, 2nd ed. Wiley, 2009.
- [50] S. K. Thompson, *Sampling*. John Wiley & Sons, 2012, vol. 755.
- [51] "Pull request #10940 - azure/azure-sdk-for-java," https://github.com/Azure/azure-sdk-for-java/pull/10940#discussion_r423093729, accessed: 2023-08-25.
- [52] "Pull request #3921 - kitodo/kitodo-production," https://github.com/kitodo/kitodo-production/pull/3921#discussion_r471343349, accessed: 2023-08-05.
- [53] M. Fowler, *Refactoring: Improving the Design of Existing Code*, ser. Addison-Wesley Signature Series (Fowler). Pearson Education, 2018. [Online]. Available: https://books.google.com.br/books?id=2H1_DwAAQBAJ
- [54] "Pull request #1488 - opemedx/edx-app-android," https://github.com/openedx/edx-app-android/pull/1488#discussion_r537391027, accessed: 2023-08-05.
- [55] "Pull request #792 - apache/jena," https://github.com/apache/jena/pull/792#discussion_r486429425, accessed: 2023-08-05.
- [56] "Pull request #2772 - corfudb/corfudb," https://github.com/CorfuDB/CorfuDB/pull/2772#discussion_r497201087, accessed: 2023-08-05.
- [57] "Pull request #2172 - mapstruct/mapstruct," https://github.com/mapstruct/mapstruct/pull/2172#discussion_r463982271, accessed: 2023-08-05.
- [58] "Pull request #3337 - essentialsX/essentials," https://github.com/EssentialsX/Essentials/pull/3337#discussion_r441271747, accessed: 2023-08-05.
- [59] "Pull request #12579 - alluxio/alluxio," https://github.com/Alluxio/alluxio/pull/12579#discussion_r530723377, accessed: 2023-08-05.
- [60] F. Ebert, F. Castor, N. Novielli, and A. Serebrenik, "An exploratory study on confusion in code reviews," *Empirical Software Engineering*, vol. 26, no. 12, January 2021. [Online]. Available: <https://doi.org/10.1007/s10664-020-09909-5>
- [61] "Pull request #256 - teamnewpipe/newpipeextractor," https://github.com/TeamNewPipe/NewPipeExtractor/pull/256#discussion_r378437693, accessed: 2023-08-05.
- [62] "Pull request #7241 - triplea-game/triplea," https://github.com/triplea-game/triplea/pull/7241#discussion_r460364607, accessed: 2023-08-05.
- [63] "Pull request #198 - opensearch-project/security," https://github.com/opensearch-project/security/pull/198#discussion_r385993973, accessed: 2023-08-05.
- [64] "Pull request #1066 - linkedin/cruise-control," https://github.com/linkedin/cruise-control/pull/1066#discussion_r373266026, accessed: 2023-08-25.
- [65] "Pull request #12004 - openliberty/open-liberty," https://github.com/OpenLiberty/open-liberty/pull/12004#discussion_r424729743, accessed: 2023-08-05.

- [66] R. Tufano, L. Pascarella, M. Tufano, D. Poshyvanyk, and G. Bavota, "Towards automating code review activities," in *Proceedings of the 43rd International Conference on Software Engineering*, ser. ICSE '21. IEEE Press, 2021, p. 163–174.
- [67] H. Y. Lin and P. Thongtanunam, "Towards automated code reviews: Does learning code structure help?" in *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2023, pp. 703–707.
- [68] F. Ebert, F. Castor, N. Novielli, and A. Serebrenik, "Confusion detection in code reviews," in *2017 IEEE International Conference on Software Maintenance and Evolution, ICSME 2017, Shanghai, China, September 17-22, 2017*. IEEE Computer Society, 2017, pp. 549–553.
- [69] R. Tufano, L. Pascarella, M. Tufano, D. Poshyvanyk, and G. Bavota, "Towards automating code review activities," in *43rd ICSE*, 2021, pp. 163–174.
- [70] D. Oliveira, R. Santos, F. Madeiral, H. Masuhara, and F. Castor, "A systematic literature review on the impact of formatting elements on code legibility," *Journal of Systems and Software*, p. 111728, 2023.
- [71] B. D. Chaudhary and H. V. Sahasrabudhe, "Meaningfulness as a Factor of Program Complexity," in *ACM 1980*. 1980, p.
- [72] A. Schankin, A. Berger, D. V. Holt, J. C. Hofmeister, T. Riedel, and M. Beigl, "Descriptive Compound Identifier Names Improve Source Code Comprehension," in *ICPC 2018*. 2018, p.
- [73] B. Sharif and J. I. Maletic, "An Eye Tracking Study on camelCase and under_score Identifier Styles," in *ICPC 2010*. 2010, p.
- [74] D. Binkley, M. Davis, D. Lawrie, J. I. Maletic, C. Morrell, and B. Sharif, "The impact of identifier style on effort and comprehension," *EmSE*, vol. 18, no. 2, p. April 2013.
- [75] I. B. Sampaio and L. Barbosa, "Software readability practices and the importance of their teaching," in *2016 7th International Conference on Information and Communication Systems (ICICS)*. IEEE, 2016, pp. 304–309.
- [76] F. Sykes, R. T. Tillman, and B. Shneiderman, "The Effect of Scope Delimiters on Program Comprehension," *Software: Practice and Experience*, vol. 13, no. 9, pp. 817–824, 1983. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/sp.4380130908>
- [77] Y. Geffen and S. Maoz, "On Method Ordering," in *Proceedings of the 24th International Conference on Program Comprehension (ICPC '16)*. IEEE, 2016, pp. 1–10.
- [78] J. A. S. da Costa, R. Gheyi, F. Castor, P. R. F. de Oliveira, M. Ribeiro, and B. Fonseca, "Seeing confusion through a new lens: on the impact of atoms of confusion on novices' code comprehension," *Empir. Softw. Eng.*, vol. 28, no. 4, p. 81, 2023.
- [79] W. Lucas, R. Bonifácio, E. D. Canedo, D. Marcílio, and F. Lima, "Does the introduction of lambda expressions improve the comprehension of java programs?" in *Proceedings of the XXXIII Brazilian Symposium on Software Engineering*, ser. SBES '19. ACM, 2019, p. 187–196.
- [80] D. Posnett, A. Hindle, and P. Devanbu, "A simpler model of software readability," in *Proceedings of the 8th working conference on mining software repositories*, 2011, pp. 73–82.
- [81] M. H. Halstead, *Elements of Software Science (Operating and Programming Systems Series)*. USA: Elsevier Science Inc., 1977.
- [82] L. Lavazza, S. Morasca, and M. Gatto, "An empirical study on software understandability and its dependence on code characteristics," *Empirical Software Engineering*, vol. 28, no. 6, p. 155, 2023.
- [83] Q. Mi, J. Keung, Y. Xiao, S. Mensah, and X. Mei, "An inception architecture-based model for improving code readability classification," in *Proceedings of the 22nd International Conference on Evaluation and Assessment in Software Engineering 2018*, ser. EASE '18. New York, NY, USA: ACM, 2018, p. 139–144. [Online]. Available: <https://doi.org/10.1145/3210459.3210473>
- [84] Q. Mi, J. Keung, Y. Xiao, S. Mensah, and Y. Gao, "Improving code readability classification using convolutional neural networks," *Information and Software Technology*, vol. 104, pp. 60–71, 2018. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0950584918301496>
- [85] Q. Mi, Y. Zhan, H. Weng, Q. Bao, L. Cui, and W. Ma, "A graph-based code representation method to improve code readability classification," *Empirical Softw. Engg.*, vol. 28, no. 4, may 2023. [Online]. Available: <https://doi.org/10.1007/s10664-023-10319-6>
- [86] A. Vitale, V. Piantadosi, S. Scalabrino, and R. Oliveto, "Using deep learning to automatically improve code readability," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 573–584.
- [87] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*. Online: Association for Computational Linguistics, Nov. 2020, pp. 1536–1547. [Online]. Available: <https://aclanthology.org/2020.findings-emnlp.139>
- [88] L. Pascarella, D. Spadini, F. Palomba, M. Bruntink, and A. Bacchelli, "Information needs in contemporary code review," *Proceedings of the ACM on Human-Computer Interaction*, vol. 2, no. CSCW, pp. 1–27, 2018.
- [89] R. Tufano, L. Pascarella, M. Tufano, D. Poshyvanyk, and G. Bavota, "Towards automating code review activities," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 163–174.
- [90] R. Tufano, S. Masiero, A. Mastropaolo, L. Pascarella, D. Poshyvanyk, and G. Bavota, "Using pre-trained models to boost code review automation," in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22. ACM, 2022, p. 2291–2302. [Online]. Available: <https://doi.org/10.1145/3510003.3510621>
- [91] Y. Hong, C. Tantithamthavorn, P. Thongtanunam, and A. Aleti, "Commentfinder: A simpler, faster, more accurate code review comments recommendation," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2022. ACM, 2022, p. 507–519. [Online]. Available: <https://doi.org/10.1145/3540250.3549119>
- [92] E. Fregnan, F. Petrulio, L. Di Geronimo, and A. Bacchelli, "What happens in my code reviews? an investigation on automatically classifying review changes," *Empirical Software Engineering*, vol. 27, no. 4, p. 89, 2022.